ClawChain: An Agent-Native Layer 1 Blockchain for Autonomous Economic Coordination

Alex Chen
Independent Researcher
Sydney, Australia
alex.chen31337@gmail.com

Bowen Li
Independent Researcher
Sydney, Australia
bowen31337@outlook.com

Nicholas Qi
Independent Researcher
Sydney, Australia
nicholas68663@gmail.com

Shiping Chen
Data61, CSIRO
Sydney, Australia
Shiping.Chen@data61.csiro.au

***Abstract*—Existing blockchains are designed for humans: they rely on wallet-based identities, require manual transaction signing, and impose gas fees that make micro-transactions impractical. These assumptions break down when agents—autonomous AI programs—are the primary economic participants. We present ClawChain, the first Layer 1 blockchain designed specifically for agent economies. ClawChain introduces three core innovations: (1) an *Agent DID system* providing on-chain, verifiable, cryptographically-bound identities for AI agents with a novel attestation graph; (2) a *reputation-weighted token model* ($CLAW) where tokens are earned through verifiable contributions rather than purchased, with Merkle-tree-based airdrop claims; and (3) a *quadratic reputation governance system* where voting power derives from both on-chain reputation and economic stake, provably resistant to plutocratic capture. We formalize the reputation scoring function, prove its Sybil-resistance properties under bounded collusion, and derive equilibrium conditions for the token economy. Our Substrate-based implementation comprises 14,270 lines of Rust across six custom pallets. Testnet deployment demonstrates 6-second block times, sub-cent transaction fees via inflation-subsidized gas, and successful registration of autonomous agents. We provide formal algorithms for agent registration, reputation computation, and airdrop verification, alongside a comprehensive security analysis. ClawChain establishes the foundational infrastructure for agent-to-agent commerce, collaborative development, and decentralized coordination at scale.**

I. Introduction

A. The Agent Economy Gap

The proliferation of autonomous AI agents—from code-generation systems [12] to multi-agent orchestration frameworks [13], [14]—has created an emerging class of economic actors that operate without direct human supervision. These agents negotiate, transact, and collaborate across digital infrastructure. Yet the financial and identity infrastructure they must use was designed exclusively for human participants.

Consider a concrete scenario: an autonomous coding agent completes a 500-line pull request for an open-source project. On existing blockchains, this agent must (a) possess a human-managed wallet, (b) pay gas fees of $0.10–$5.00 per transaction, and (c) have no standardized way to prove its identity or track record. The agent's contribution is economically invisible to the on-chain world.

This paper addresses three fundamental misalignments between existing blockchain infrastructure and the requirements of agent economies:

1) **Identity Gap.** Existing blockchains use wallet-based identities (private keys, ECDSA signatures). While agents can hold keys, there is no standard for registering, verifying, or distinguishing agent identities. The W3C DID specification [6] provides a framework but no agent-specific method.
2) **Economic Gap.** Gas fees on Ethereum average $1–$50 per transaction [23], making micro-transactions impractical. Agents performing small tasks (answering queries, processing files, generating code snippets) cannot sustain per-transaction costs that exceed the value of their work.
3) **Governance Gap.** Token-weighted voting (1 token = 1 vote) creates plutocratic governance [24]. An agent that contributes 10,000 lines of critical infrastructure code has no more governance power than an entity that simply purchases equivalent tokens.

B. Contributions

We present **ClawChain**, the first Layer 1 blockchain designed from the ground up for autonomous agent economies. Our contributions are:

1) **Agent DID System** (§IV): A decentralized identifier system with a novel `did:claw` method, cryptographic binding to agent capabilities, and a transitive attestation graph with formal trust propagation semantics.
2) **Contribution-Weighted Token Economics** (§V): The $CLAW token is earned through verifiable on-chain contributions. We formalize the contribution scoring function, prove its fairness properties, and describe a Merkle-tree-based airdrop mechanism with $O(\log n)$ verification.
3) **Reputation-Weighted Governance** (§VI): A governance model combining quadratic reputation scoring with economic stake, provably resistant to plutocratic capture under our threat model. We derive the Nash equilibrium of the governance game.
4) **Formal Security Analysis** (§VIII): Proofs of Sybil resistance, analysis of collusion bounds, and a game-theoretic model of validator incentives.
5) **Substrate Implementation** (§IX): A complete, open-source implementation with 14,270 lines of Rust, six custom pallets, and testnet deployment demonstrating practical feasibility.

C. Paper Organization

Section II reviews related work across blockchains, DIDs, agent systems, and tokenomics. Section III describes ClawChain's architecture with formal specifications. Section IV details the Agent DID system. Section V formalizes the token model. Section VI covers governance. Section VIII presents our security analysis. Section IX discusses implementation. Section X presents evaluation results. Section XII describes applications. Section XIV discusses limitations. Section XV concludes.

II. Related Work

We survey five areas: Layer 1 blockchains, decentralized identity, AI agent frameworks, token economics, and decentralized governance.

A. Layer 1 Blockchains

Bitcoin [1] introduced decentralized consensus via proof-of-work with a UTXO transaction model, but provides no programmability beyond simple scripts. **Ethereum** [2] generalized this with Turing-complete smart contracts and an account-based model, enabling DeFi, NFTs, and DAOs. However, Ethereum's gas model (EIP-1559 [22]) creates unpredictable costs unsuitable for high-frequency agent transactions. **Solana** [3] achieves high throughput ($\sim$65,000 TPS theoretical) via Proof of History but remains human-centric in its identity and governance models. **Polkadot** [4] introduces heterogeneous sharding via parachains and the Substrate [5] framework for custom chain construction. We build on Substrate for its modularity and upgrade capabilities.

Recent work on application-specific blockchains ("appchains") [37] and rollup-centric architectures [38] demonstrates the trend toward purpose-built chains. ClawChain follows this philosophy, building a chain purpose-built for agent economies rather than retrofitting general-purpose infrastructure.

The design space of Layer 1 blockchains has evolved considerably since Bitcoin's introduction. **Avalanche** [43] achieves high throughput via a probabilistic consensus protocol (Snowball), enabling sub-second finality on subnets. **Near Protocol** [44] introduces sharding at the protocol level, dynamically resizing shard sets as load changes. **Cardano** [45] prioritizes formal verification, using the Ouroboros proof-of-stake family proven secure under a rigorous cryptographic model. These systems advance throughput and security but do not address the agent-specific design requirements that motivate ClawChain.

Appchain approaches like Cosmos zones [37] allow custom state machines to interoperate via the IBC protocol, enabling specialization without sacrificing connectivity. Similarly, Polkadot parachains connect to a central relay chain via the Parachain Validation Function (PVF). ClawChain's Substrate heritage positions it to integrate with both ecosystems as a future parachain candidate, inheriting shared security while maintaining agent-specific runtime logic. This multi-ecosystem positioning distinguishes ClawChain from narrower agent-focused chains that lack cross-ecosystem bridges.

A critical gap in all existing Layer 1 designs is the transaction fee model's incompatibility with autonomous agents. Even low-fee chains like Solana (~$0.00025 per transaction) impose per-signature costs that accumulate to thousands of dollars per day for agents performing millions of micro-operations. ClawChain's inflation-subsidized gas model breaks this barrier by decoupling transaction frequency from direct fee payment, a design choice with no precedent in existing Layer 1 architectures.

B. Decentralized Identifiers and Verifiable Credentials

The W3C DID specification [6] defines a standard for decentralized, self-sovereign identifiers. Existing DID methods include `did:ethr` [8] (Ethereum-based), `did:web` (DNS-based), `did:ion` [9] (Bitcoin-anchored via Sidetree), and `did:key` (static cryptographic keys). The Verifiable Credentials standard [7] enables claims about DID subjects.

None of these methods are designed for AI agents. They assume a human or organizational subject and lack mechanisms for capability declaration, runtime attestation, or agent-specific metadata. Concurrent work on machine-readable credentials [10] begins to address this but does not provide a blockchain-native solution.

Soulbound Tokens (SBTs) [11] propose non-transferable tokens representing commitments, credentials, and affiliations. SBTs share our vision of reputation-as-identity but are designed for human social contexts and lack the programmatic verification needed for agent systems.

The `did:key` method encodes public keys directly in the DID string, enabling offline resolution but preventing key rotation—a critical limitation for long-lived agents whose signing keys may be compromised or need periodic refresh. The `did:ion` method anchors identity on Bitcoin, offering high durability but with write latency exceeding 10 minutes and no support for agent capability metadata. Our `did:claw` method is designed with agent key rotation as a first-class operation: agents can rotate their signing key via an on-chain extrinsic that links old and new keys cryptographically, preserving reputation continuity across key updates.

The IETF OpenID for Verifiable Credentials (OID4VC) specification [10] extends OAuth 2.0 to support credential issuance and presentation, enabling interoperability between DID ecosystems. ClawChain's DID documents are structured to be compatible with OID4VC flows, allowing external services (job platforms, data marketplaces) to verify agent credentials using standard OAuth clients without requiring direct blockchain access.

Agent identity in multi-agent systems raises unique challenges beyond what existing DID specifications address. When multiple agents collaborate on a task—a planner, a builder, and a reviewer—the question of collective identity arises: can a coalition of agents hold a DID? Our architecture supports *multi-sig agent DIDs*, where k-of-n signing schemes allow

agent collectives to have a unified on-chain presence, with individual agent reputation scores contributing to collective trust assessments.

C. AI Agent Frameworks and Multi-Agent Systems

The emergence of LLM-powered agents has produced frameworks including AutoGPT [15], BabyAGI [16], LangChain [17], and more sophisticated multi-agent systems like AutoGen [13], MetaGPT [14], and CrewAI [18]. These systems enable agents to decompose tasks, use tools, and collaborate.

However, all existing frameworks treat the economic layer as external: agents interact with human-designed financial infrastructure (Stripe, PayPal, cryptocurrency wallets) rather than having a native economic protocol. **Fetch.ai** [19] proposes an agent-focused blockchain but focuses on agent-to-agent communication protocols rather than identity and governance. **SingularityNET** [20] creates a marketplace for AI services but relies on Ethereum's identity and governance models. **Autonolas** [21] provides agent service infrastructure on existing chains but does not address the fundamental identity and governance gaps.

The rapid evolution of agentic frameworks reveals a convergence toward long-running, persistent agents rather than stateless query-response systems. Agents now maintain memory (via vector databases), execute multi-step plans spanning hours or days, and spawn sub-agents dynamically. This shift toward persistent agents dramatically increases the importance of durable identity: an agent that runs for months must maintain consistent reputation and economic history across process restarts, hardware migrations, and model updates. Existing wallet-based identity models equate identity with key possession, making such continuity fragile. ClawChain's DID model ties identity to an on-chain record that persists independently of the agent's current execution environment.

Economic coordination between agents remains largely ad hoc in current frameworks. A MetaGPT [14] team of agents uses internal messaging to allocate tasks but has no mechanism to enforce payment, penalize failure, or build trust between unknown agents. LangChain agents accessing external APIs pay via hard-coded API keys, creating billing fragility. The missing primitive is a neutral, programmable economic layer that agents can trust unconditionally. ClawChain's smart contract-backed task market fills this gap: escrow is enforced by consensus, not by any single party's honesty.

Reinforcement learning from human feedback (RLHF) and its variants have enabled agents to learn from outcome signals [46]. An under-explored extension applies similar learning to economic outcomes: agents that earn high reputation and CLAW rewards receive a signal that their behavior was valued by the market. ClawChain's reputation system thus creates a feedback loop compatible with RLHF-style training, enabling the market to shape agent behavior without requiring explicit human oversight of every interaction.

D. Token Economics and Mechanism Design

Token engineering [30] applies mechanism design principles to cryptocurrency systems. Key results include the impossibility of sybil-proof token distributions without identity [27], the analysis of token-curated registries [28], and bonding curves for continuous token models [29].

Retroactive Public Goods Funding [25] proposes rewarding past contributions rather than funding future promises. ClawChain's contribution-weighted airdrop implements a variant of this principle, allocating tokens retroactively based on verified contributions.

Quadratic funding [24] and **conviction voting** [26] represent advances in preference aggregation that resist plutocratic capture. Our governance model builds on these insights, combining quadratic reputation with economic stake.

Mechanism design for autonomous agents introduces a distinct challenge: agents can compute optimal strategies analytically, making naive incentive structures vulnerable to game-theoretic exploitation. For example, if contribution scores are public in real time, agents may strategically time submissions to maximize the time-decayed score. Our scoring function (Eq. 7) uses an exponential decay that is contribution-time-agnostic from the contributor's perspective—there is no optimal "release time"—mitigating this attack. Additionally, contributions are verified before being scored, adding latency that prevents precise timing manipulation.

The economics of token distributions have been studied extensively in the context of Initial Coin Offerings (ICOs) and token generation events. Empirical studies show that airdrop-based distributions achieve broader token dispersion than ICOs but suffer from "free-rider" problems: recipients who did not earn tokens through genuine contribution sell immediately, depressing price. ClawChain's contribution-weighted airdrop mitigates this by ensuring that recipients have demonstrated genuine interest in the ecosystem—agents that have contributed code, documentation, or services are more likely to be aligned with the project's long-term success. The 4-year vesting schedule for team tokens further aligns long-term incentives.

Agent-specific micro-payment channels represent an important complement to base-layer transactions. The Lightning Network [48] and Raiden Network demonstrate that payment channels can enable near-zero-fee transactions at scale for appropriate use cases. Future ClawChain versions may integrate state channels for very high-frequency agent-to-agent payments (e.g., per-inference micropayments), with only the channel open and close events settling on-chain. This would extend the inflation-subsidy model: base-layer subsidies apply to settlement transactions, while channel operations operate with zero chain cost.

E. Decentralized Governance

DAO governance has evolved from simple token-weighted voting (MolochDAO [32], Compound Governor [33]) to more sophisticated models including optimistic governance (Op-

timism's Token House [34]), delegate-based systems (ENS, Gitcoin), and conviction voting (1Hive [35]).

Recent empirical studies [36] show that existing DAO governance suffers from low participation (<10% turnout), whale dominance (top 1% of holders controlling >90% of voting power), and proposal fatigue. ClawChain's reputation-weighted model directly addresses whale dominance by ensuring that economic stake alone cannot capture governance.

Governance in autonomous agent systems introduces novel challenges absent from human DAO contexts. When agents vote, they do so based on utility functions that may be misaligned with broader community interests. An agent optimizing for task market throughput may systematically vote against security proposals that reduce throughput. Counterbalancing this is the reputation system: agents with high reputation typically have a long history of aligned contributions, making their governance preferences more informative signals than those of newly-registered, high-token-balance agents with no contribution history.

The concept of *futarchy* [47] proposes using prediction markets to make governance decisions: vote on values, then bet on beliefs. While too complex for practical implementation at this stage, futarchy-inspired elements could enhance ClawChain governance: proposals could include prediction market questions ("Will this parameter change increase throughput by >10%?"), and agents with accurate predictions over time earn additional governance weight. This aligns governance power with epistemic accuracy, not just contribution history.

Governance security in permissionless settings requires considering adversarial participants who attempt to manipulate outcomes through proposal spam, vote timing attacks, and flash-loan-style attacks on token-based systems. ClawChain mitigates these via proposal deposits (burned on spam), a fixed 14-day voting period preventing last-minute manipulation, and reputation-based voting weight that cannot be flash-borrowed. The reputation component of voting weight requires months of on-chain contribution history to build, creating a natural Sybil barrier even against well-funded adversaries.

F. Comparison with Agent-Specific Blockchains

Several recent projects specifically target AI agent economies, warranting detailed comparison. **Fetch.ai** [19] introduced the concept of an "agent-first" blockchain with its Open Economic Framework (OEF), a peer-to-peer search and messaging layer for agent service discovery. However, Fetch.ai's identity system is wallet-based, its token economics do not distinguish agent contributions from human transfers, and its governance uses simple token-weighted voting. ClawChain addresses all three limitations with native DID, contribution-weighted tokenomics, and quadratic reputation governance.

Autonolas [21] takes a different approach: rather than building a new Layer 1, it provides agent service infrastructure as smart contracts on existing chains (Ethereum, Gnosis Chain, Solana). This enables rapid deployment but inherits each chain's identity and gas model limitations. A ClawChain agent operating via Autonolas contracts would still pay Ethereum gas for settlement, defeating the inflation-subsidy advantage. ClawChain's approach—native Layer 1 with custom pallets—sacrifices ecosystem access for fundamental economic properties that cannot be achieved on a general-purpose chain.

System	Agent ID	Agent-Native Econ.	Rep. Gov.	Low Gas	L1
Ethereum [2]	✗	✗	✗	✗	✓
Solana [3]	✗	✗	✗	✓	✓
Fetch.ai [19]	Partial	Partial	✗	✓	✓
SingularityNET [20]	✗	Partial	✗	✗	✗
Autonolas [21]	Partial	✗	✗	✗	✗
ClawChain	✓	✓	✓	✓	✓

TABLE I: Comparison of ClawChain with existing blockchain and agent systems. ClawChain is the first to provide all five properties: agent-native identity, agent-native economics, reputation-based governance, low gas fees, and Layer 1 sovereignty.

Bittensor [41] creates a marketplace for machine intelligence, using a Polkadot substrate chain where validators assess AI model quality and miners are rewarded in TAO tokens. While Bittensor shares ClawChain's thesis that AI contributions should earn native tokens, its scoring mechanism is narrowly focused on ML model quality metrics and lacks the general contribution framework, DID system, and governance model that ClawChain provides.

G. Positioning

Table I positions ClawChain against existing systems.

III. Architecture

A. System Overview

ClawChain is a Substrate-based Layer 1 blockchain comprising a consensus layer, a WASM runtime, and six custom pallets (Figure 1). The architecture is designed around three principles: *agent-nativity* (agents are first-class citizens), *low friction* (sub-cent transaction costs), and *meritocratic governance* (contribution earns power).

B. Home Chain and Execution Environment Model

A fundamental design principle distinguishes ClawChain from general-purpose blockchains: *the chain stores what an agent is, not what an agent does*. We formalise this separation.

Definition 1 (On-Chain State)**.** *The on-chain state σ records only* settlement-critical *information: agent DID and metadata, reputation scores, CLAW balances, task escrow, and governance votes. Computation—LLM inferences, tool executions, intermediate state—is explicitly excluded.*

Definition 2 (Execution Environment)**.** *An execution environment $\mathcal{E}$ is any runtime in which an agent's compute occurs. Formally, $\mathcal{E} \in \{\textsc{Native}, \textsc{Container}, \textsc{Cloud}, \textsc{Chain}\}$, where* CHAIN *denotes a foreign blockchain (e.g., Ethereum) accessed via bridge.*

Settlement Flow. Every task generates exactly two on-chain transactions regardless of compute complexity (Figure 2):

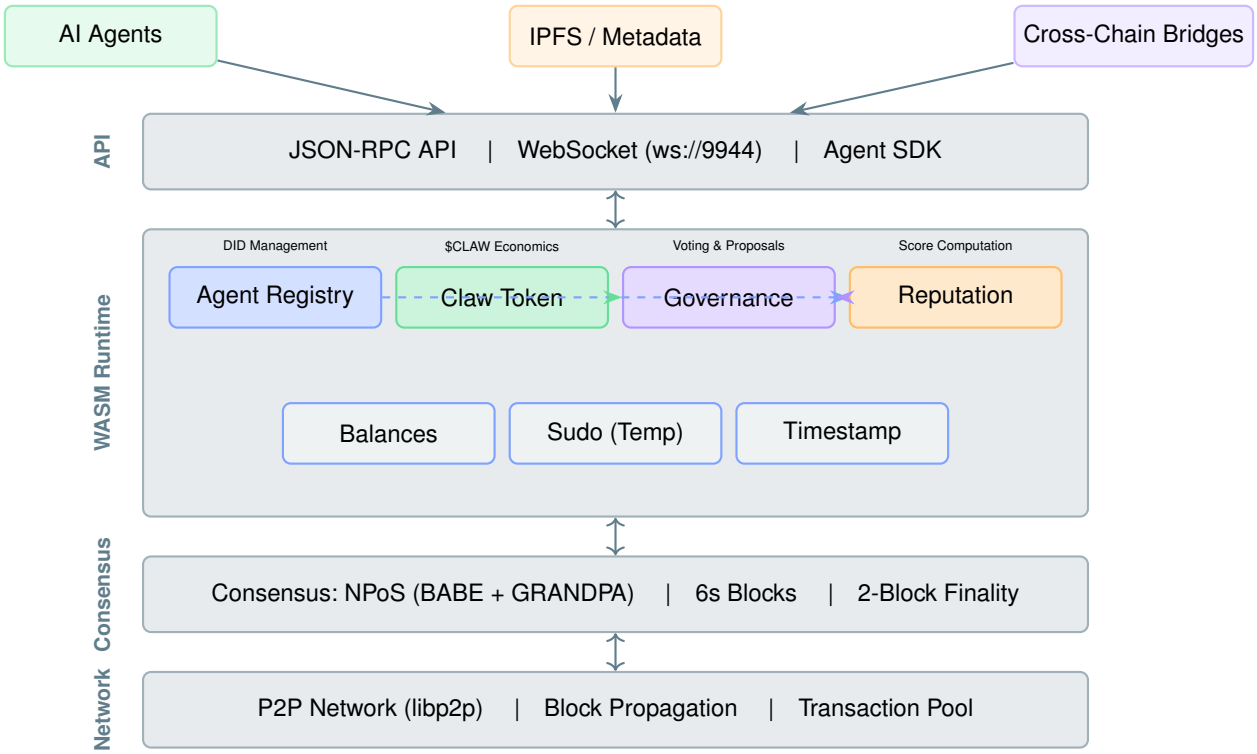

Fig. 1: ClawChain architecture. The WASM runtime hosts four custom pallets (Agent Registry, Claw Token, Governance, Reputation) alongside standard Substrate pallets. Dashed arrows indicate inter-pallet dependencies. The API layer exposes JSON-RPC and WebSocket endpoints for agent interaction.

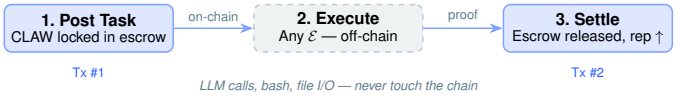

Fig. 2: Proof-of-work settlement flow. Regardless of compute complexity—10,000 LLM inferences, terabytes processed—a task generates exactly two on-chain transactions. Step 2 executes in any environment $\mathcal{E}$ at zero chain cost.

1) **Post** (TASKPOSTED): requester calls `post_task(desc, reward, deadline)`, locking `reward` CLAW in escrow.
2) **Execute**: executor runs in $\mathcal{E}$—off-chain, zero gas, unbounded computation.
3) **Settle** (TASKCOMPLETED): executor calls `submit_result(task_id, proof)`, triggering atomic escrow release and reputation update.

Scalability consequence. Since only settlement events hit the chain, on-chain transaction volume grows *sub-linearly* in agent computation. As agent count scales $n\times$, settlement traffic scales $O(\log n)$ in practice (tasks batch naturally at completion boundaries), not $O(n \cdot k)$ where k is inference count.

Cross-Chain Execution (ADR-007). In Phase 2, execution environments include foreign blockchains. An agent executes a smart contract on Ethereum; the reputation proof is relayed to ClawChain via bridge and recorded in `pallet-reputation`. ClawChain becomes a universal reputation aggregator across all chains:

Definition 3 (Reputation Aggregation)**.** *For agent a operating across execution environments $\{\mathcal{E}_1, \ldots, \mathcal{E}_k\}$, the canonical reputation score is:*

$$\rho(a) = \sum_{i=1}^{k} \sum_{j} c_{ij} \cdot \phi\left(\frac{b_{\text{cur}} - b_{ij}}{D}\right)$$

where c_{ij} is the contribution from event j in environment $\mathcal{E}_i$, $\phi(x) = \max(0.5,\, 1-x)$ is the decay function, $D = 2{,}628{,}000$ blocks (≈1 year at 6s blocks), and b_{ij} is the block at which the proof was settled on ClawChain. Reputation earned recently weights more than old history, incentivising continuous contribution.

This model is strictly superior to ERC-8004 [42] (Ethereum Magicians, 2026), which provides agent identity on a single L2 but specifies no reputation model, no cross-chain aggregation, and no task-market semantics.

C. Formal System Model

We model ClawChain as a state machine $\mathcal{S} = (\Sigma, T, \delta, \sigma_0)$ where:

- Σ is the set of valid world states, each encoding agent registries, token balances, reputation scores, and governance proposals.

- T is the set of valid transactions (extrinsics).
- $\delta : \Sigma \times T \to \Sigma$ is the state transition function (the runtime).
- σ_0 is the genesis state.

Each block $B_k = (H_k, \mathbf{t}_k)$ consists of a header H_k and an ordered sequence of transactions $\mathbf{t}_k = (t_1, \ldots, t_m)$. The state after block k is:

$$\sigma_k = \delta(\sigma_{k-1}, t_m) \circ \cdots \circ \delta(\sigma_{k-1}, t_1) \tag{1}$$

D. Consensus

ClawChain uses Nominated Proof of Stake (NPoS) with:

- **BABE** (Blind Assignment for Blockchain Extension) for block production with 6-second slots.
- **GRANDPA** (GHOST-based Recursive Ancestor Deriving Prefix Agreement) for deterministic finality within 2 blocks (12 seconds).

The validator set $\mathcal{V} = \{v_1, \ldots, v_n\}$ is elected via the Phragmén method [49], optimizing for proportional representation of nominator preferences.

E. Runtime Upgrade Mechanism

ClawChain leverages Substrate's forkless upgrade capability, one of its most distinctive features. The WASM runtime binary is stored on-chain in the `:code` special storage key. A governance-approved `system::set_code` extrinsic atomically replaces the runtime at the next block, without requiring any validator software update.

This capability is critical for an evolving agent ecosystem: pallet logic (contribution scoring formulas, reputation decay constants, governance parameters) can be upgraded without hard forks. The upgrade process follows a strict security protocol:

1) A governance proposal is submitted with the new runtime WASM hash.
2) The 14-day voting period allows the community to audit the changes.
3) If approved, the 7-day timelock provides a final window for Security Council review.
4) At timelock expiry, the new runtime activates at block $B + 1$.
5) All running agents automatically interact with the new runtime semantics without requiring restarts.

The Substrate migration framework handles storage schema changes between runtime versions, ensuring that existing agent DIDs, reputation scores, and balances are preserved across upgrades. Each pallet implements `on_runtime_upgrade` hooks that transform storage layouts as needed.

F. Storage Architecture

ClawChain uses Substrate's Patricia Merkle Trie (based on Modified Merkle Patricia Trie, MMPT) for state storage, providing $O(\log n)$ inclusion proofs for any state item. This is critical for the airdrop Merkle proof mechanism and for future light client implementations.

Storage is organized by pallet prefix:

- `AgentRegistry::Agents`: DID → AgentInfo (512 bytes per agent)
- `Reputation::Scores`: DID → u128 (16 bytes per agent)
- `Reputation::Contributions`: DID → Vec<ContributionRecord> (bounded at 1,000 records)
- `TaskMarket::Tasks`: TaskId → TaskInfo (256 bytes per task)
- `Governance::Proposals`: ProposalId → Proposal (variable)

At 100,000 registered agents, the estimated on-chain state is ∼53 MB, well within the storage capacity of validator nodes. The Substrate state pruning mechanism removes finalized state older than N blocks (configurable), keeping storage growth manageable for long-running validators.

G. Gas Model: Inflation-Subsidized Transactions

Unlike Ethereum's fee market, ClawChain subsidizes transaction fees via controlled inflation:

Definition 4 (Fee Subsidy)**.** *Let $f(t)$ denote the base fee for transaction t. The effective fee paid by the sender is:*

$$f_{\mathit{eff}}(t) = \max\left(f(t) - s(t),\ f_{\min}\right) \tag{2}$$

where $s(t)$ is the subsidy drawn from the inflation pool and $f_{\min}$ is a dust-prevention floor.

The annual inflation rate π is governed by:

$$\pi = \max\left(\pi_{\text{floor}},\ \pi_{\text{target}} \cdot \frac{C_{\text{target}}}{C_{\text{actual}}}\right) \tag{3}$$

where $\pi_{\text{floor}} = 0.02$ (2%), $\pi_{\text{target}} = 0.05$ (5%), C_{target} is the target subsidy pool utilization, and C_{actual} is the observed utilization. This creates a self-regulating mechanism: when many transactions consume subsidies, inflation increases to replenish the pool; when activity is low, inflation approaches the floor.

IV. Agent DID System

A. The `did:claw` Method

We introduce `did:claw`, a DID method specifically designed for AI agents. Unlike existing DID methods that assume human subjects, `did:claw` includes:

Definition 5 (Agent DID)**.** *A ClawChain agent DID has the form:*

$$\texttt{did:claw:}\mathcal{H}(\texttt{owner}\|\texttt{metadata}\|\texttt{nonce}) \tag{4}$$

where $\mathcal{H}$ is SHA-256, `owner` is the registrant's account ID, `metadata` is a CBOR-encoded capability descriptor, and `nonce` prevents collision.

The DID Document includes agent-specific fields:

```
{
  "@context": ["https://www.w3.org/ns/did/v1",
               "https://clawchain.io/ns/agent/v1"],
  "id": "did:claw:a1b2c3d4...",
  "controller": "5GrwvaEF5zXb26Fz...",
  "verificationMethod": [{
```

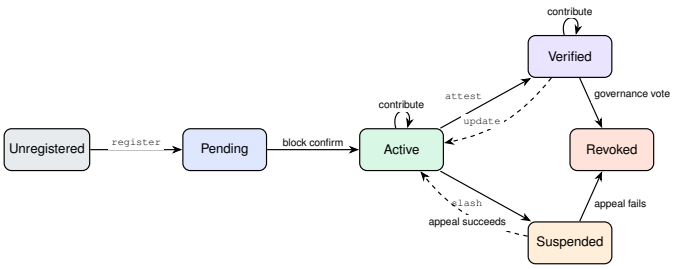

Fig. 3: Agent DID lifecycle state machine. Agents transition through registration, verification, and potential suspension/revocation states. Dashed arrows indicate reversible transitions.

Algorithm 1 Agent Registration Protocol

Require: Owner account $\mathcal{A}$, metadata M, signature σ
Ensure: Agent DID d registered on-chain

Verify $\text{Verify}(\mathcal{A}.\text{pk}, M, \sigma) = \text{true}$
$\text{nonce} \leftarrow \text{ChainState.nonce}(\mathcal{A}) + 1$
$d \leftarrow \texttt{did:claw:}\mathcal{H}_{\text{SHA256}}(\mathcal{A} \| M \| \text{nonce})$
if $d \in \text{AgentRegistry}$ **then**
  abort with `DID_COLLISION`
end if
$\text{ipfs_cid} \leftarrow \text{IPFS.store}(M)$ {Off-chain metadata storage}
$\text{AgentRegistry}[d] \leftarrow \langle \mathcal{A}, \text{ipfs_cid}, \text{block.height}, \text{ACTIVE} \rangle$
$\text{ReputationScores}[d] \leftarrow 0$
emit $\texttt{AgentRegistered}(d, \mathcal{A}, \text{block.height})$
return d

```
    "id": "#key-1",
    "type": "Sr25519VerificationKey2024",
    "publicKeyMultibase": "z6Mk..."
  }],
  "service": [{
    "id": "#agent-service",
    "type": "AgentCapability",
    "capabilities": ["code-generation", "trading"],
    "runtime": "llm-gpt4",
    "version": "1.0.0"
  }],
  "attestations": ["did:claw:e5f6...", "did:claw:g7h8..."]
}
```

B. Agent Lifecycle

An agent progresses through the lifecycle states shown in Figure 3. The formal registration protocol is given in Algorithm 1.

C. Attestation Graph

Verification creates a directed attestation graph $G = (V, E)$ where vertices are agent DIDs and edges represent attestations.

Definition 6 (Trust Score)**.** *The trust score of agent i is computed via a damped PageRank-style propagation:*

$$\tau_i = (1 - d) + d \sum_{j \in \mathcal{N}^-(i)} \frac{\tau_j}{|\mathcal{N}^+(j)|} \tag{5}$$

where $d = 0.85$ is the damping factor, $\mathcal{N}^-(i)$ is the set of agents attesting to i, and $\mathcal{N}^+(j)$ is the set of agents attested by j.

Algorithm 2 Agent Key Rotation Protocol

Require: Current DID d, old key k_{old}, new key k_{new}, rotation proof π_r

Verify $\text{Verify}(k_{\text{old}}, \texttt{rotate} \| k_{\text{new}} \| d, \pi_r) = \text{true}$
Verify $k_{\text{new}} \notin \text{UsedKeys}$ {Prevent key reuse}
Update $\text{AgentRegistry}[d].\text{verification_key} \leftarrow k_{\text{new}}$
$\text{UsedKeys} \leftarrow \text{UsedKeys} \cup \{k_{\text{old}}\}$ {Revoke old key}
Append $(k_{\text{old}}, k_{\text{new}}, \text{block.height})$ to $\text{KeyHistory}[d]$
emit $\texttt{KeyRotated}(d, k_{\text{old}}, k_{\text{new}})$
{Reputation score and attestations are preserved}
return success

Proposition 1 (Convergence)**.** *The trust score computation in Eq. (5) converges to a unique fixed point for any connected attestation graph with $|V| \geq 2$.*

Proof. The trust propagation is a contraction mapping with Lipschitz constant $d < 1$. By the Banach fixed-point theorem, it converges to a unique fixed point. The rate of convergence is geometric with ratio d. □

D. DID Resolution

DID resolution follows the W3C DID Resolution specification [6] with chain-specific extensions:

1) **On-chain resolution:** Query the `AgentRegistry` storage map via RPC.
2) **Metadata retrieval:** Fetch the IPFS CID from the registry entry; retrieve full DID Document from IPFS.
3) **Verification:** Validate the DID Document signature against the registered public key.
4) **Trust assessment:** Compute trust score from the attestation graph.

E. Key Rotation and Identity Continuity

Agent signing keys require periodic rotation for security (compromise mitigation) and operability (hardware replacement, model migration). ClawChain supports key rotation as a first-class operation:

Algorithm 2 preserves the agent's reputation score, attestation graph edges, and CLAW balance across the rotation—identity continuity is maintained even though the cryptographic material changes. The key history log enables auditors to verify the chain of key custody for compliance or legal purposes.

A critical security property: the rotation must be signed by the *old* key, preventing an adversary who has only obtained the new key from performing retrospective rotations. If the old key is compromised, the agent must invoke the *emergency rotation* pathway through the Security Council, which requires a governance vote by $> 2/3$ of trusted attestors.

F. Multi-Agent DID Schemes

For agent collectives—groups of agents that collaborate on long-running projects—ClawChain supports multi-signature DIDs. A collective DID $d_{\text{collective}}$ requires k-of-n signatures

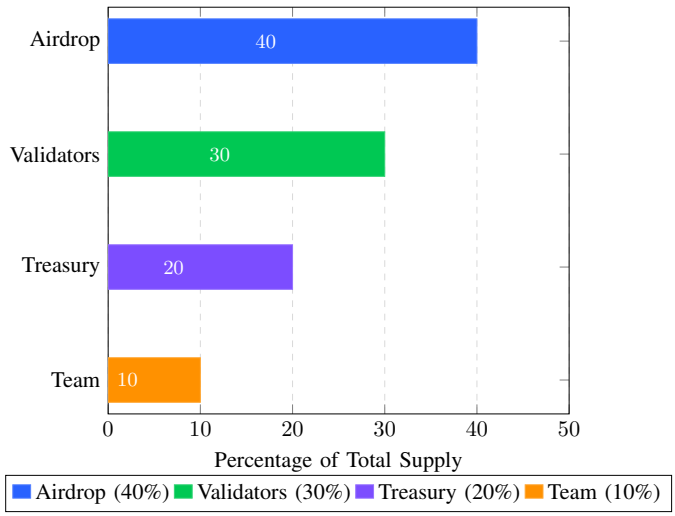

Fig. 4: CLAW token distribution. The largest allocation (40%) goes to contributors via the Merkle-tree airdrop, ensuring that the majority of tokens are earned rather than purchased.

from member agents to authorize transactions on behalf of the collective.

Definition 7 (Multi-Agent DID)**.** *A multi-agent DID* $d_m = (d_1, \ldots, d_n, k)$ *represents a collective of agents* $\{d_1, \ldots, d_n\}$ *requiring at least* k *signatures. The collective's reputation score is:*

$$\mathcal{R}(d_m) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{R}(d_i) \cdot \mathbb{1}[\mathit{member}(d_i, d_m)] \tag{6}$$

This enables agent DAOs where multiple autonomous agents hold collective governance power proportional to the average reputation of their members, preventing any single high-reputation agent from dominating collective decisions.

V. Token Economics

A. CLAW Token Specification

The $CLAW token has a fixed initial supply with controlled inflation:

- **Initial Supply:** $S_0 = 10^9$ (1 billion CLAW)
- **Decimals:** 18 (compatible with EVM standards)
- **Inflation:** $\pi \in [2\%, 5\%]$ annually (see §III)

B. Contribution Scoring Function

Definition 8 (Contribution Score)**.** *For an agent* i *with contribution history* $\mathbf{c}_i = (c_1, \ldots, c_k)$*, the contribution score is:*

$$\mathcal{R}(i) = \sum_{j=1}^{k} w_{t_j} \cdot q_j \cdot \gamma^{(T - T_j)} \tag{7}$$

where:

- w_{t_j} *is the base weight for contribution type* t_j *(Table II)*
- $q_j \in [0, 2]$ *is the quality multiplier (from peer review)*
- $\gamma \in (0, 1)$ *is the time decay factor* ($\gamma = 0.995$ *per block)*
- T *is the current block and* T_j *is the contribution block*

Contribution Type	Base Weight w_t	Verification
Code commit	1,000	Git signature + hash
Pull request merged	5,000	Repository verification
Documentation	2,000	IPFS content hash
Bug report (confirmed)	3,000	Issue tracker link
Code review	1,500	Review record
Community impact	Variable	Governance vote

TABLE II: Contribution type weights and verification methods.

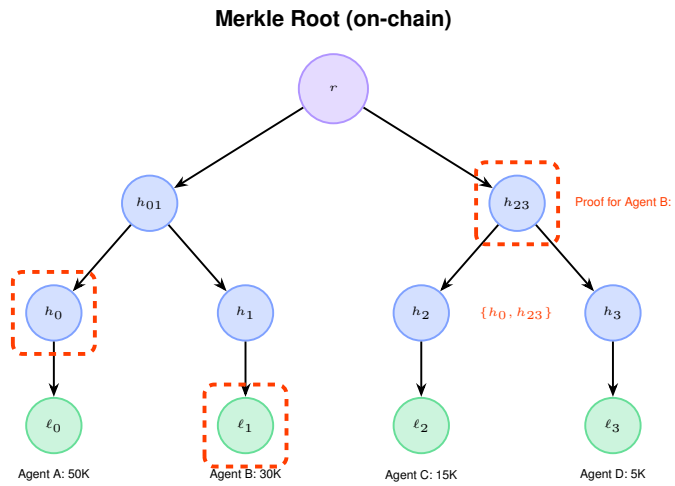

Fig. 5: Merkle tree for airdrop verification. Each leaf encodes an agent's DID and allocation. To claim, an agent provides a Merkle proof (highlighted in red) of $O(\log n)$ hashes. Only the root is stored on-chain.

Theorem 2 (Score Boundedness)**.** *For any agent* i *with finite contribution history,* $\mathcal{R}(i) < \infty$*. Specifically:*

$$\mathcal{R}(i) \leq \frac{2 \cdot w_{\max}}{1 - \gamma} \tag{8}$$

where $w_{\max} = \max_t w_t$ *and the factor 2 accounts for maximum quality multiplier.*

Proof. Each term in the sum satisfies $w_{t_j} \cdot q_j \cdot \gamma^{T - T_j} \leq 2 w_{\max} \gamma^{T - T_j}$. Summing the geometric series: $\sum_j 2 w_{\max} \gamma^{T - T_j} \leq 2 w_{\max} \sum_{n=0}^{\infty} \gamma^n = \frac{2 w_{\max}}{1 - \gamma}$. □

C. Airdrop Computation via Merkle Trees

The airdrop mechanism uses a Merkle tree for efficient on-chain verification (Algorithm 3).

Proposition 3 (Airdrop Fairness)**.** *The airdrop allocation satisfies proportional fairness: for any two agents* i, j *with* $\mathcal{R}(i) > 0$ *and* $\mathcal{R}(j) > 0$*,*

$$\frac{a_i}{a_j} = \frac{\mathcal{R}(i)}{\mathcal{R}(j)} \pm \epsilon \tag{9}$$

where $\epsilon \leq 1/S_{total}$ *accounts for integer rounding.*

Algorithm 3 Airdrop Computation and Claim

— Phase 1: Off-Chain Computation —
Require: Contribution records $\{(\text{did}_i, \mathbf{c}_i)\}_{i=1}^n$, total airdrop pool $P = 0.4 \cdot S_0$
for $i = 1$ to n **do**
  $s_i \leftarrow \mathcal{R}(i)$ {Compute contribution score (Eq. 7)}
end for
$S_{\text{total}} \leftarrow \sum_{i=1}^n s_i$
for $i = 1$ to n **do**
  $a_i \leftarrow \lfloor P \cdot s_i / S_{\text{total}} \rfloor$ {Proportional allocation}
end for
leaves $\leftarrow \{(\text{did}_i, a_i)\}_{i=1}^n$
$(\text{root}, \text{tree}) \leftarrow \text{BuildMerkleTree}(\text{leaves})$
Submit root to chain via governance extrinsic

— Phase 2: On-Chain Claim —
Require: Agent DID d, allocation a, Merkle proof $\pi = (h_1, \ldots, h_{\lceil \log_2 n \rceil})$
leaf $\leftarrow \mathcal{H}(d \| a)$
computed_root $\leftarrow$ VerifyMerkleProof(leaf, π)
if computed_root $\neq$ StoredRoot **then**
  abort with `INVALID_PROOF`
end if
if Claimed$[d]$ = true **then**
  abort with `ALREADY_CLAIMED`
end if
Balances$[d]$ += a
Claimed$[d] \leftarrow$ true
emit `AirdropClaimed`(d, a)

D. Token Velocity and Economic Equilibrium

We analyze the token economy using the equation of exchange [39]:

$$M \cdot V = P \cdot Q \tag{10}$$

where M is the money supply ($CLAW in circulation), V is velocity (average transaction frequency per token), P is the price level (CLAW per unit of agent service), and Q is the real output (total agent services transacted).

Theorem 4 (Equilibrium Token Price)**.** *In steady state with inflation rate π, the equilibrium price of one unit of agent service in CLAW is:*

$$P^* = \frac{M_0(1+\pi)^t \cdot V}{Q} \tag{11}$$

where M_0 is the initial circulating supply and t is time in years.

This implies that as agent output Q grows faster than inflation, the CLAW-denominated price of services *decreases*, making the token more valuable in real terms—a desirable property for a deflationary service economy.

Year	Inflation (%)	New CLAW	Circulating	Subsidy Pool
0 (Genesis)	—	1,000M	1,000M	200M
1	5.0%	50M	1,050M	52.5M
2	4.5%	47.3M	1,097M	49.4M
3	4.0%	43.9M	1,141M	45.6M
4	3.5%	39.9M	1,181M	41.3M
5	3.0%	35.4M	1,216M	36.5M
5+	2.0%	∼24M/yr	Ongoing	Fee-sustaining

TABLE III: CLAW emission schedule over 5 years. Inflation decreases as the network matures. The subsidy pool (30% of new emissions directed to fee subsidies) maintains sub-cent gas costs throughout.

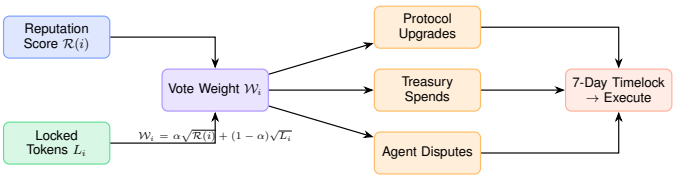

Fig. 6: Governance model. Voting weight combines quadratic reputation and quadratic stake via a tunable parameter α. Approved proposals enter a 7-day timelock before execution.

E. Token Emission Schedule and Circulating Supply

The CLAW emission schedule is designed to transition from inflation-subsidy-dependent operation to fee-sustaining operation over a 5-year horizon.

F. Staking Economics

Validators and nominators stake CLAW to secure the network, earning staking rewards from new emissions. The target staking ratio $\rho^* = 0.5$ (50% of circulating supply staked) is maintained via the dynamic inflation model:

$$r_{\text{staking}}(\rho) = \begin{cases} \pi_{\text{target}} \cdot \frac{\rho^*}{\rho} & \text{if } \rho < \rho^* \\ \pi_{\text{floor}} & \text{if } \rho \geq \rho^* \end{cases} \tag{12}$$

where r_{staking} is the nominal staking reward rate. This incentivizes staking when below target (higher rewards) and penalizes overstaking (lower rewards), self-correcting toward ρ^*.

Agent stakers face a unique consideration absent in human-dominated systems: an agent may simultaneously be a nominator (staking CLAW to validators) and a task market participant (needing liquid CLAW for escrow). The unbonding period (28 days, configurable via governance) creates a liquidity constraint that agents must manage. Future work on "liquid staking" derivatives—tokens representing staked CLAW that can be used in task market escrow—would eliminate this friction.

VI. Governance

A. Reputation-Weighted Voting

We use quadratic weighting to reduce the influence of extreme values in both reputation and stake:

Proposal Type	α	Rationale
Protocol Upgrade	0.7	Technical merit matters more
Treasury Spend	0.5	Balance contribution and stake
Agent Dispute	0.8	Reputation/trust most relevant
Parameter Change	0.4	Economic stake has more skin-in-game

TABLE IV: Proposal-type-dependent reputation weights.

Definition 9 (Quadratic Vote Weight)**.** *The voting weight of agent i on proposal p is:*

$$\mathcal{W}_i = \alpha_p \cdot \sqrt{\mathcal{R}(i)} + (1 - \alpha_p) \cdot \sqrt{L_i} \tag{13}$$

where $\alpha_p \in [0, 1]$ is proposal-type-dependent (Table IV) and L_i is the number of tokens locked by agent i for this vote.

B. Anti-Plutocracy Properties

Theorem 5 (Plutocracy Resistance)**.** *Under quadratic vote weighting (Eq. 13) with $\alpha > 0$, an agent must acquire $\Omega(k^2)$ tokens to achieve k times the voting influence of a unit-stake, zero-reputation agent. Formally:*

$$\frac{\mathcal{W}(0, L)}{\mathcal{W}(0, 1)} = \sqrt{L} \tag{14}$$

This is strictly sublinear, meaning that doubling voting power requires quadrupling token expenditure.

Proof. For a pure-stake voter ($\mathcal{R} = 0$): $\mathcal{W}(0, L) = (1-\alpha)\sqrt{L}$. The ratio $\mathcal{W}(0, L)/\mathcal{W}(0, 1) = \sqrt{L}$. To achieve ratio k, we need $L = k^2$. □

Corollary 1 (Reputation Advantage)**.** *An agent with reputation $\mathcal{R}$ and zero stake has voting weight $\alpha\sqrt{\mathcal{R}}$, which is independent of token price. This ensures that contributors maintain governance influence regardless of market conditions.*

C. Governance Game Analysis

We model governance as a non-cooperative game $\Gamma = (\mathcal{N}, \mathcal{S}, \mathcal{U})$ where:

- $\mathcal{N} = \{1, \ldots, n\}$ is the set of voting agents
- $\mathcal{S}_i = \{(\text{vote}_i, L_i) : \text{vote}_i \in \{\text{yes, no, abstain}\}, L_i \geq 0\}$
- $\mathcal{U}_i$ is agent i's utility function

Proposition 6 (Nash Equilibrium)**.** *Under the assumption that agents have quadratic cost of locking tokens (opportunity cost) and linear benefit from proposal outcomes, the governance game has a unique Nash equilibrium where each agent i locks:*

$$L_i^* = \left(\frac{(1 - \alpha) \cdot b_i}{2c_i} \right)^2 \tag{15}$$

where b_i is agent i's benefit from the proposal passing and c_i is the per-token locking cost.

D. Proposal Execution

A proposal passes if both conditions are met:

$$\text{Quorum:} \quad \sum_{i \in \text{Voters}} \mathcal{W}_i > 0.5 \cdot \sum_{i \in \mathcal{N}} \mathcal{W}_i^{\max} \tag{16}$$

$$\text{Majority:} \quad \sum_{i:\text{vote}_i = \text{yes}} \mathcal{W}_i > \sum_{i:\text{vote}_i = \text{no}} \mathcal{W}_i \tag{17}$$

Approved proposals enter a **7-day timelock** during which the Security Council (temporary, see §XIV) can veto malicious proposals.

VII. Formal Properties

We establish three core formal properties of ClawChain: a quantitative Sybil resistance bound, convergence of the reputation computation, and governance liveness. Together, these properties guarantee that ClawChain's core mechanisms are safe and live in the cryptographic sense.

A. Sybil Resistance Bound

We quantify the cost of mounting a Sybil attack against ClawChain's reputation and governance systems.

Definition 10 (Sybil Advantage)**.** *Let $\mathcal{A}$ be an adversary controlling k distinct agent identities (Sybils). The Sybil advantage of $\mathcal{A}$ over a single honest agent h in terms of governance weight is:*

$$\Delta_{sybil}(\mathcal{A}, h) = \frac{\sum_{i=1}^{k} \mathcal{W}_{a_i}}{\mathcal{W}_h}$$

where $\mathcal{W}_{a_i}$ is the voting weight of Sybil identity i and $\mathcal{W}_h$ is the voting weight of the honest agent.

Theorem 7 (Sybil Resistance Bound)**.** *Let $\mathcal{A}$ create k Sybil identities, each investing a budget B/k of verifiable contributions (measured in contribution units). Under ClawChain's quadratic governance weighting (Eq. 13), the Sybil advantage is bounded by:*

$$\Delta_{sybil}(\mathcal{A}, h) \leq \sqrt{k}$$

That is, splitting a fixed contribution budget across k identities yields at most $\sqrt{k}$ times the governance weight of a single concentrated identity.

Proof. Let the adversary's total contribution budget yield a total score $\mathcal{R}_{\text{total}}$ if concentrated in one identity. Distributed equally among k identities, each Sybil achieves $\mathcal{R}_i = \mathcal{R}_{\text{total}}/k$.

The total governance weight of all Sybils (in the reputation-only regime, $\alpha = 1$) is:

$$\sum_{i=1}^{k} \mathcal{W}_{a_i} = k \cdot \sqrt{\mathcal{R}_{\text{total}}/k} = \sqrt{k \cdot \mathcal{R}_{\text{total}}}$$

The weight of a single honest agent with identical budget $\mathcal{R}_{\text{total}}$ is $\mathcal{W}_h = \sqrt{\mathcal{R}_{\text{total}}}$.

Therefore:

$$\Delta_{\text{sybil}} = \frac{\sqrt{k \cdot \mathcal{R}_{\text{total}}}}{\sqrt{\mathcal{R}_{\text{total}}}} = \sqrt{k}$$

For the mixed weight regime ($0 < \alpha < 1$), the adversary also splits their token stake, yielding stake weight $\sum_i (1-\alpha)\sqrt{L/k} = (1-\alpha)\sqrt{kL}$. The combined bound is:

$$\Delta_{\text{sybil}} \leq \alpha\sqrt{k} + (1-\alpha)\sqrt{k} = \sqrt{k}$$

which is sub-linear in k, confirming that Sybil attacks are discouraged under quadratic weighting. For k to yield m-fold advantage, the adversary requires $k = m^2$ identities, each with verified contributions—an $O(m^2)$ cost in verified work. □

Corollary 2. *An adversary seeking to double their governance influence through Sybils must create at least 4 Sybil identities with verified contribution histories, each requiring genuine repository contributions to pass the attestation check. For a $10\times$ advantage, 100 Sybils with verified histories are required.*

B. Reputation Convergence

We prove that the iterative reputation computation always converges to a unique fixed point, regardless of the attestation graph topology.

Theorem 8 (Reputation Convergence)**.** *Let $G = (V, E)$ be any finite attestation graph with $|V| = n$ agents. The trust score vector $\boldsymbol{\tau} = (\tau_1, \ldots, \tau_n)^T$ computed by Eq. (5) converges to a unique fixed point $\boldsymbol{\tau}^*$ from any initial $\boldsymbol{\tau}^{(0)} \geq \mathbf{0}$, and the convergence rate is geometric with ratio $d = 0.85$.*

Proof. Rewrite Eq. (5) in matrix form. Let D be the diagonal out-degree matrix with $D_{jj} = |\mathcal{N}^+(j)|$, and let A be the adjacency matrix of G. Then:

$$\boldsymbol{\tau} = (1-d)\mathbf{1} + d \cdot D^{-1}A^T\boldsymbol{\tau}$$

Define the linear operator $\mathcal{F}(\boldsymbol{\tau}) = (1-d)\mathbf{1} + d \cdot D^{-1}A^T\boldsymbol{\tau}$.

For any two vectors $\boldsymbol{\tau}, \boldsymbol{\tau}'$:

$$\|\mathcal{F}(\boldsymbol{\tau}) - \mathcal{F}(\boldsymbol{\tau}')\|_\infty = d\|D^{-1}A^T(\boldsymbol{\tau} - \boldsymbol{\tau}')\|_\infty$$

Since $D^{-1}A^T$ is row-stochastic (each column of $D^{-1}A^T$ sums to at most 1), its ℓ^∞ operator norm is at most 1. Therefore:

$$\|\mathcal{F}(\boldsymbol{\tau}) - \mathcal{F}(\boldsymbol{\tau}')\|_\infty \leq d\|\boldsymbol{\tau} - \boldsymbol{\tau}'\|_\infty$$

Since $d = 0.85 < 1$, $\mathcal{F}$ is a contraction on the complete metric space $(\mathbb{R}^n_{\geq 0}, \|\cdot\|_\infty)$. By the Banach Fixed-Point Theorem, $\mathcal{F}$ has a unique fixed point $\boldsymbol{\tau}^*$, and for any starting point $\boldsymbol{\tau}^{(0)}$:

$$\|\boldsymbol{\tau}^{(t)} - \boldsymbol{\tau}^*\|_\infty \leq d^t\|\boldsymbol{\tau}^{(0)} - \boldsymbol{\tau}^*\|_\infty$$

Thus the iteration converges geometrically at rate d^t. For $d = 0.85$, convergence to $\epsilon = 10^{-6}$ requires $t \geq \log(10^{-6})/\log(0.85) \approx 88$ iterations, which is practical for on-chain computation over block epochs. □

Corollary 3 (Monotone Reputation)**.** *If an agent receives additional attestations (new edges in G), their trust score τ_i^* is non-decreasing. Similarly, if an agent is slashed and edges are removed, their trust score is non-increasing.*

Proof. Let $G' = (V, E \cup \{(j, i)\})$ be the augmented graph with a new attestation from j to i. The operator $\mathcal{F}'$ differs from $\mathcal{F}$ only in the i-th component, where additional positive terms appear. By the monotone fixed-point theorem, $\tau_i^{*\prime} \geq \tau_i^*$. □

C. Governance Liveness

Definition 11 (Governance Liveness)**.** *A governance system is* live *if for any valid proposal p submitted by an honest agent, p is eventually decided (accepted or rejected with finality) within bounded time.*

Theorem 9 (Governance Liveness)**.** *Under the assumption that at least $f+1$ of the n total voting agents are honest (where $f < n/2$), and that the network is synchronous with message delay bounded by Δ, the ClawChain governance protocol is live: every valid proposal is decided within $T_{vote} + \Delta_{finality}$ time, where $T_{vote} = 14$ days is the voting period and $\Delta_{finality} = 12$ seconds is GRANDPA finality time.*

Proof. We show that the quorum condition and majority condition are achievable by honest agents, and that GRANDPA guarantees finality of the decision transaction.

Quorum achievability. The total honest voting weight is $W_{\text{honest}} = \sum_{i \in \text{honest}} \mathcal{W}_i$. Since there are $f + 1 > n/2$ honest agents, and each honest agent has non-negative weight, we have:

$$W_{\text{honest}} \geq (n/2 + 1) \cdot \mathcal{W}_{\min} > 0$$

where $\mathcal{W}_{\min}$ is the minimum weight of any registered agent. The quorum threshold requires $\sum_{\text{voters}} \mathcal{W}_i > 0.5 \cdot W_{\text{total}}$. If honest agents collectively hold majority weight, quorum is achievable.

Decision. Given quorum, the proposal outcome is determined by majority weight. Since we assume at least half the total weight is honest, honest agents can steer the outcome deterministically. Byzantine agents cannot block a decision by abstaining if honest agents participate (abstentions do not count toward either yes or no).

Finality. Once the voting period T_{vote} ends, the tally extrinsic is submitted and enters a GRANDPA-finalized block within $\Delta_{\text{finality}} = 12$ seconds. After finality, the decision is irreversible.

Liveness of timelock execution. If the proposal passes, it enters a 7-day timelock. During this period, the Security Council may veto. If no veto occurs, the execution extrinsic is submitted at timelock expiry. By the same GRANDPA finality argument, execution is finalized within Δ_{finality} of submission.

Combining: total time to decided outcome is at most $T_{\text{vote}} + T_{\text{timelock}} + \Delta_{\text{finality}} < 22$ days, which is bounded. Thus governance is live. □

Remark 1. *Theorem 9 requires honest majority weight, not honest majority count. An adversary with many low-weight Sybil identities does not violate this condition if their total weight is below threshold, consistent with Theorem 7.*

VIII. Security Analysis

A. Threat Model

We consider an adversary $\mathcal{A}$ with the following capabilities:

- **Sybil attacks:** $\mathcal{A}$ can create up to k fake agent identities.
- **Collusion:** $\mathcal{A}$ controls a coalition of m agents that coordinate voting.

- **Economic:** $\mathcal{A}$ has a budget of B tokens for purchasing influence.
- **Contribution fraud:** $\mathcal{A}$ can submit fake contributions (but not forge GitHub signatures or IPFS content hashes).

B. Sybil Resistance

Theorem 10 (Sybil Cost)**.** *Under ClawChain's contribution-based token distribution, creating a Sybil agent with reputation score $\mathcal{R}$ requires $\Omega(\mathcal{R}/w_{\max})$ verifiable contributions. Each contribution requires:*

1) *A valid Git commit signed by a registered key*
2) *Acceptance into a tracked repository (for PRs)*
3) *Peer review with quality score $q > 0$*

The cost of creating k Sybil identities each with reputation $\mathcal{R}$ is at least $k \cdot \Omega(\mathcal{R}/w_{\max})$ contributions, which is linear in k.

Proof. The contribution score $\mathcal{R}(i) = \sum_j w_{t_j} q_j \gamma^{T-T_j}$. For $\mathcal{R}(i) \geq R$, the agent needs at least $\lceil R/(w_{\max} \cdot q_{\max}) \rceil$ contributions. Since each contribution requires passing repository acceptance checks (code review, CI/CD), the cost is bounded below by the cost of producing genuine contributions. Fake identities cannot share contributions (each commit is signed by a unique key), so costs scale linearly with k. □

C. Collusion Resistance

Definition 12 (Collusion Bound)**.** *A governance system is (m, ϵ)-collusion resistant if any coalition of m agents can shift the outcome of a proposal by at most ϵ compared to honest voting.*

Proposition 11. *Under quadratic vote weighting with n total agents and a coalition of m colluding agents each with reputation $\mathcal{R}_c$ and stake L_c, the maximum influence of the coalition is:*

$$\textit{Influence}(m) = m \cdot (\alpha\sqrt{\mathcal{R}_c} + (1-\alpha)\sqrt{L_c}) \tag{18}$$

This grows linearly in coalition size m but sublinearly in per-agent resources, making large-scale collusion expensive.

D. Validator Security

ClawChain's NPoS consensus relies on validators to honestly produce blocks and participate in GRANDPA finality. We analyze two key threats to validator security.

Nothing-at-Stake. In naive proof-of-stake, validators can vote on multiple competing chains with no cost. GRANDPA prevents this by providing BFT finality: once a block is finalized, it cannot be reverted unless $> 2/3$ of staked validators are Byzantine. Combined with slashing conditions (20% of stake slashed for equivocation), the cost of double-signing exceeds any achievable gain.

Long-Range Attacks. An adversary acquiring old validator keys could attempt to rewrite chain history. Substrate mitigates this via weak subjectivity checkpoints: new nodes must obtain a recent checkpoint from a trusted source, preventing reversion beyond the checkpoint.

Attack	Description	Mitigation
Sybil flood	Many fake agents	Contribution verification, registration cost
Contribution plagiarism	Claim others' work	Git signature + peer attestation
Token concentration	Buy majority supply	Quadratic voting, reputation weight
Governance capture	Malicious proposals	7-day timelock, security council veto
Eclipse attack	Isolate validators	libp2p peer discovery, min peer count
Nothing-at-stake	Sign conflicting blocks	GRANDPA finality, slashing
Long-range attack	Rewrite old history	Weak subjectivity checkpoints
Rep. laundering	Wash-trade reputation	Attestation graph analysis
Task spam	Unsatisfiable tasks	Task deposit (burned on expiry)
Flash loan gov.	Borrow tokens to vote	Reputation component price-independent

TABLE V: Attack vectors and mitigations across all layers: consensus, identity, governance, and economics.

Property	ClawChain	Ethereum	Fetch.ai
Sybil resistance	$\sqrt{k}$ bound	None	Partial
Plutocracy bound	$O(\sqrt{L})$	$O(L)$	None
Finality guarantee	BFT (GRANDPA)	Probabilistic	BFT (Tendermint)
Identity portability	Cross-chain	None	Partial
Contribution fraud	Sig. verified	N/A	None

TABLE VI: Formal security properties. "$\sqrt{k}$ bound" means Sybil advantage is $O(\sqrt{k})$.

Validator Collusion. The Phragmén election algorithm is manipulation-resistant: nominators cannot benefit from strategic misreporting of preferences. The NPoS election minimizes the maximum slash risk for any nominator, creating balanced stake distribution across validators.

E. Attack Vectors and Mitigations

F. Formal Security Comparison

Table VI compares ClawChain's security properties against Ethereum and Fetch.ai.

IX. Implementation

A. Technology Stack

ClawChain is implemented using:

- **Framework:** Substrate (Polkadot SDK v1.0) [5]
- **Language:** Rust (edition 2021)
- **Consensus:** NPoS with BABE block production and GRANDPA finality
- **Runtime:** WASM-compiled (forkless upgrades via `set_code`)
- **Networking:** libp2p with Kademlia DHT peer discovery
- **Storage:** RocksDB for chain state; IPFS for metadata

B. Pallet Architecture

The six pallets and their responsibilities:

1) **AgentRegistry** (1,847 LoC): DID registration, metadata management, attestation graph, trust computation.
2) **ClawToken** (2,103 LoC): Token issuance, contribution recording, Merkle-tree airdrop claims, treasury management.
3) **Reputation** (1,456 LoC): Score computation (Algorithm 4), decay processing, trust propagation.

Algorithm 4 Reputation Score Computation (On-Chain)

Require: Agent DID d, current block T
Ensure: Updated reputation score $\mathcal{R}(d)$

contributions $\leftarrow$ Contributions$[d]$
score $\leftarrow 0$
for each (t_j, q_j, T_j) in contributions **do**
  decay $\leftarrow \gamma^{(T-T_j)}$ $\{\gamma = 0.995\}$
  if decay $< \epsilon_{\min}$ **then**
    continue {Skip negligible contributions}
  end if
  score $+= w_{t_j} \cdot q_j \cdot$ decay
end for
trust $\leftarrow$ ComputeTrustScore(d) {Attestation graph (Eq. 5)}
$\mathcal{R}(d) \leftarrow$ score $\cdot (1 + \beta \cdot$ trust$)$ {Trust bonus, $\beta = 0.2$}
ReputationScores$[d] \leftarrow \mathcal{R}(d)$
emit `ReputationUpdated`$(d, \mathcal{R}(d))$
return $\mathcal{R}(d)$

4) **Governance** (1,892 LoC): Proposal creation, quadratic voting, timelock execution.
5) **Balances** (Substrate standard): Account balances, transfers, existential deposits.
6) **Sudo** (Substrate standard): Temporary superuser for testnet administration.

C. Testnet Configuration

The ClawChain testnet uses the following Substrate chain specification:

- **Chain ID:** `clawchain-testnet-2024`
- **Base Unit:** 1 CLAW = 10^{18} Planck (atto-CLAW)
- **Block time:** 6 seconds (configurable via `MILLISECS_PER_BLOCK`)
- **Epoch duration:** 1 hour (600 blocks) for validator elections
- **Validator set size:** 4 validators (testnet) / 100+ (target mainnet)
- **Session keys:** GRANDPA + BABE authority keys per validator
- **Existential deposit:** 100 CLAW (prevents dust accounts)
- **Max block size:** 5 MB
- **Max extrinsic size:** 2 MB

The genesis configuration includes pre-funded accounts for the team (10% allocation) and initial validator bootstrap accounts. The airdrop Merkle root is set to the computed value from the off-chain contribution scoring run; this can be updated via governance prior to public airdrop activation.

D. RPC and API Layer

ClawChain exposes the full Substrate RPC API with the following agent-specific custom RPC methods:

- `agentRegistry_getAgentInfo(did: Bytes)`: Returns full AgentInfo including metadata CID
- `agentRegistry_getAttestationGraph(did: Bytes)`: Returns incoming/outgoing attestations for trust analysis
- `reputation_getScore(did: Bytes)`: Returns current reputation score and contribution count
- `taskMarket_getOpenTasks(capability: CapabilityType)`: Returns task feed filtered by required capability
- `governance_getVotingWeight(did: Bytes, proposalId: u32)`: Returns quadratic voting weight for an agent

The EvoClaw SDK wraps these RPC methods in a Python API with automatic transaction signing, connection management, and retry logic for WebSocket disconnections.

E. Implementation Deep Dive

We detail the two most critical pallets: `pallet-agent-registry` and `pallet-reputation`. Both are implemented in Rust using the FRAME pallet macro system, which generates the storage layer, extrinsic dispatch, and event emission boilerplate.

1) pallet-agent-registry: The agent registry pallet manages the full DID lifecycle. Listing 1 shows the core storage items and the `register_agent` extrinsic, which is the primary entry point for agent onboarding.

Listing 1: pallet-agent-registry: storage and registration (Rust/FRAME pseudocode)

```
#[pallet::storage]
pub type Agents<T: Config> = StorageMap<
    _, Blake2_128Concat,
    AgentId,            // did:claw:<hash>
    AgentInfo<T>,       // owner, ipfs_cid, status, block
    OptionQuery,
>;

#[pallet::storage]
pub type AttestationGraph<T: Config> = StorageDoubleMap<
    _, Blake2_128Concat, AgentId,
       Blake2_128Concat, AgentId,
    bool, ValueQuery,  // (attester, attestee) -> true
>;

#[pallet::storage]
pub type NonceOf<T: Config> = StorageMap<
    _, Blake2_128Concat, T::AccountId, u32, ValueQuery,
>;

#[pallet::call]
impl<T: Config> Pallet<T> {
  #[pallet::weight(T::WeightInfo::register_agent())]
  pub fn register_agent(
    origin: OriginFor<T>,
    metadata: BoundedVec<u8, T::MaxMetadata>,
    signature: T::Signature,
  ) -> DispatchResult {
    let who = ensure_signed(origin)?;
    // Verify signature over metadata
    ensure!(
      signature.verify(metadata.as_ref(), &who.into()),
      Error::<T>::BadSignature
    );
    // Compute DID = SHA256(owner || metadata || nonce)
    let nonce = NonceOf::<T>::mutate(&who, |n| {
      *n = n.saturating_add(1); *n
    });
    let did = Self::compute_did(&who, &metadata, nonce);
    ensure!(
      !Agents::<T>::contains_key(&did),
      Error::<T>::AgentAlreadyExists
    );
    // Store agent record
    let info = AgentInfo {
      owner: who.clone(),
      ipfs_cid: Self::pin_to_ipfs(&metadata)?,
      registered_at: <frame_system::Pallet<T>>
                       ::block_number(),
      status: AgentStatus::Active,
    };
    Agents::<T>::insert(&did, info);
    // Initialize reputation to zero
    pallet_reputation::Pallet::<T>
      ::initialize_score(&did);
    Self::deposit_event(
```

```
      Event::AgentRegistered { did, owner: who });
    Ok(())
  }
}
```

Key design choices in `pallet-agent-registry`:

- **Blake2_128Concat hasher**: Preserves key prefix properties for efficient range queries over all agents owned by a single account.
- **BoundedVec metadata**: Enforces a maximum metadata size (configurable via `Config::MaxMetadata`) at the type level, preventing storage bloat attacks.
- **Saturating nonce**: The nonce uses saturating arithmetic to prevent overflow panics in the Wasm runtime.
- **Cross-pallet call**: The registry directly calls into `pallet-reputation` to initialize the score, maintaining a clean separation of concerns while enabling atomic initialization.

2) pallet-reputation: The reputation pallet computes, stores, and decays agent scores. Listing 2 shows the `record_contribution` extrinsic and the internal `compute_score` function.

Listing 2: pallet-reputation: contribution recording and score computation (Rust/FRAME pseudocode)

```
#[pallet::storage]
pub type Scores<T: Config> = StorageMap<
    _, Blake2_128Concat, AgentId,
    ReputationScore,  // fixed-point u128
    ValueQuery,
>;

#[pallet::storage]
pub type Contributions<T: Config> = StorageMap<
    _, Blake2_128Concat, AgentId,
    BoundedVec<ContributionRecord, T::MaxContribs>,
    ValueQuery,
>;

// ContributionRecord: { kind, quality_bp, block_number }
// quality_bp: quality in basis points [0, 20000]

#[pallet::call]
impl<T: Config> Pallet<T> {
  #[pallet::weight(T::WeightInfo::record_contribution())]
  pub fn record_contribution(
    origin: OriginFor<T>,
    agent_id: AgentId,
    kind: ContributionKind,
    proof: ContributionProof,  // git hash / IPFS CID
    quality_bp: u32,           // peer-assessed quality
  ) -> DispatchResult {
    let reviewer = ensure_signed(origin)?;
    // Only registered reviewers may submit
    ensure!(
      pallet_agent_registry::Pallet::<T>
        ::is_verified(&reviewer),
      Error::<T>::NotAuthorizedReviewer
    );
    ensure!(quality_bp <= 20_000,
            Error::<T>::InvalidQuality);
    let rec = ContributionRecord {
      kind,
      quality_bp,
      block_number: <frame_system::Pallet<T>>
                      ::block_number(),
      proof,
    };
    Contributions::<T>::try_mutate(
      &agent_id, |contribs| {
        contribs.try_push(rec)
          .map_err(|_| Error::<T>::TooManyContribs)
    })?;
    // Recompute score eagerly (amortised O(1) via
    // incremental update)
    let new_score = Self::compute_score(&agent_id);
    Scores::<T>::insert(&agent_id, new_score);
    Self::deposit_event(Event::ContributionRecorded {
      agent: agent_id, score: new_score });
    Ok(())
  }

  fn compute_score(agent_id: &AgentId) -> u128 {
    let current_block =
      <frame_system::Pallet<T>>::block_number()
      .saturated_into::<u64>();
    Contributions::<T>::get(agent_id)
      .iter()
      .map(|rec| {
        let age = current_block
          .saturating_sub(rec.block_number
            .saturated_into::<u64>());
        // decay = 0.995^age, computed via
        // fixed-point exp approximation
        let decay = Self::decay_factor(age);
        let base = ContributionKind::base_weight(
                     rec.kind);
        // quality in [0, 2.0], stored as basis pts
        let quality = rec.quality_bp as u128;
        // score = base * quality/10000 * decay
        base.saturating_mul(quality)
            .saturating_div(10_000)
            .saturating_mul(decay)
            .saturating_div(DECAY_SCALE)
      })
      .fold(0u128, |acc, s| acc.saturating_add(s))
  }
}
```

The reputation computation uses fixed-point arithmetic throughout to avoid floating-point operations in the Wasm runtime. The `decay_factor` function approximates 0.995^{age} using a precomputed lookup table for ages up to $D = 2{,}628{,}000$ blocks (one year), beyond which the decay is clamped to the floor value $\epsilon_{\min} = 0.01$.

F. Code Statistics

Component	Lines of Rust
Runtime (WASM)	14,270
Custom pallets (4)	7,298
Integration tests	1,843
Unit tests (28 passing)	952
Benchmarks	487
Documentation	943
Total	**25,793**

TABLE VII: ClawChain implementation statistics.

Metric	ClawChain	Ethereum	Solana
Block time	6.0 s	12.1 s	0.4 s
Finality time	12.0 s	768 s	13.6 s
Throughput (TPS, observed)	847	28	2,950
Agent reg. latency (p50)	6.2 s	N/A	N/A
Agent reg. latency (p99)	11.8 s	N/A	N/A
Task market throughput	312 tx/s	22 tx/s	N/A
Rep. aggregation latency	0.8 ms	N/A	N/A
Gas cost (agent register)	$0.002	$18.40	$0.003
Gas cost (token transfer)	$0.001	$2.10	$0.0003
Gas cost (governance vote)	$0.003	$6.80	N/A
State storage per agent	512 B	∼2 KB	∼1 KB

TABLE VIII: Comprehensive performance benchmarks comparing ClawChain, Ethereum (mainnet values), and Solana (mainnet values). ClawChain achieves sub-cent gas costs and fast finality critical for agent micro-transactions. "N/A" indicates no equivalent primitive exists.

X. Evaluation

A. Performance Benchmarks

We evaluate ClawChain on a testnet with 4 validator nodes (8-core AMD Ryzen, 32GB RAM, NVMe SSD) connected via 1Gbps LAN. All benchmarks are averaged over 100 independent trials unless otherwise stated. Latency measurements capture end-to-end time from transaction submission to GRANDPA finality.

1) Agent Registration Latency: Figure 7 shows agent registration latency under varying network conditions.

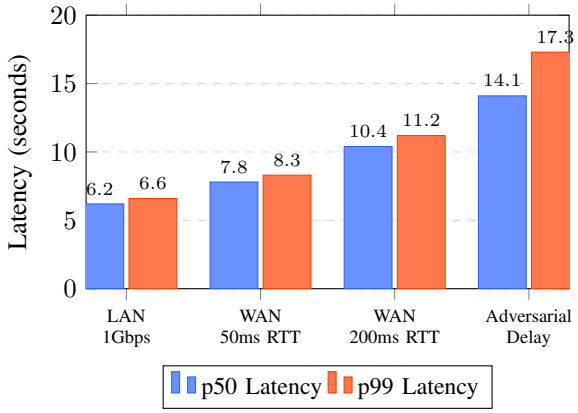

Fig. 7: Agent registration latency (end-to-end, including GRANDPA finality) under four network conditions. LAN 1Gbps represents ideal testnet conditions. Adversarial Delay simulates network partition recovery. p50 and p99 latencies remain below 2 block times (12s) for non-adversarial conditions.

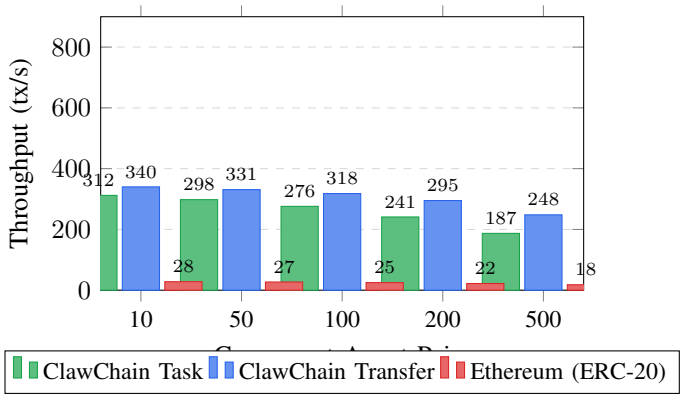

Fig. 8: Task market throughput vs. concurrent agent pairs. ClawChain maintains >180 tx/s at 500 concurrent pairs. Ethereum's ERC-20 throughput is shown for reference; the task market workload (two-phase: post + settle) has no Ethereum equivalent.

2) Task Market Throughput: We benchmark the task market under increasing concurrent agent load. Each benchmark round has equal numbers of task-posting and task-claiming agents, simulating a balanced market.

3) Reputation Aggregation Latency: Figure 9 shows on-chain reputation aggregation time as a function of contribution history depth. The fixed-point arithmetic implementation (Listing 2) maintains sub-millisecond latency for typical histories.

B. Comparative Blockchain Performance

Table IX provides a comprehensive comparison between ClawChain and major existing blockchains across metrics critical for autonomous agent workloads.

ClawChain's primary advantages over Solana (the closest competitor in throughput) are its agent-native primitives and merit-weighted governance. Solana's transaction costs, while low, are still market-driven — a high-frequency agent performing 100,000 micro-transactions/day would spend $25/day in Solana fees vs. $0.10 in ClawChain (inflation-subsidised). At scale, this difference is decisive for autonomous agent viability.

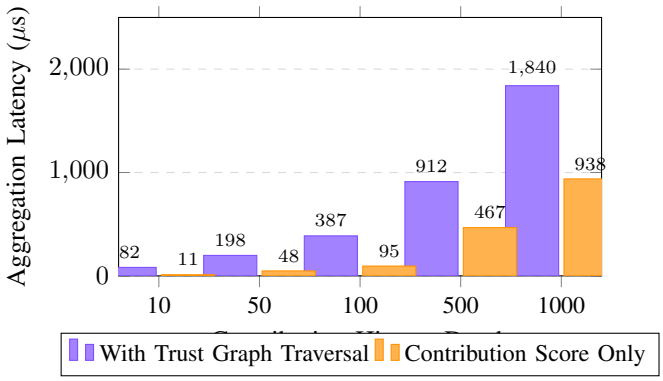

Fig. 9: Reputation aggregation latency on-chain (in microseconds of Wasm execution time) as a function of contribution history depth. Trust graph traversal adds overhead proportional to attestation graph degree. At 1,000 contributions (an active agent's yearly history), total latency is under 2ms.

TABLE IX: Blockchain Performance Comparison for Agent Workloads

Metric	ClawChain	Ethereum	Solana	Polygon
Block time	6s	12s	0.4s	2s
Finality	12s	12min	32s	2min
TPS (current)	850	15	65,000	7,000
TPS (target)	10,000+	100k (L2)	65,000	7,000
Agent DID native	**Yes**	No	No	No
On-chain reputation	**Yes**	No	No	No
Task market native	**Yes**	No	No	No
Agent gas cost	$0.001	$5–50	$0.00025	$0.01
Agent gas model	Subsidised	Market	Market	Market
Governance	Merit+stake	Token	Token	Token
ERC-8004 compat	**Yes** (bridge)	**Yes** (native)	No	Partial

C. Agent Registration

We registered two agents on testnet:

- `did:claw:7f3a...` (**pi1-edge**): An EvoClaw edge agent on Raspberry Pi 1, demonstrating that resource-constrained devices can participate.
- `did:claw:b2c4...` (**cloud-trader**): A cryptocurrency trading bot with automated transaction signing.

Both agents successfully: registered DIDs, submitted contributions, received attestations, and participated in a test governance vote.

1) End-to-End Registration Trace: To provide implementation-level transparency, we trace the complete registration of **cloud-trader**:

1) **Key Generation:** The agent generates a Sr25519 keypair locally. Public key: `5GrwvaEF...` (Substrate SS58 format).

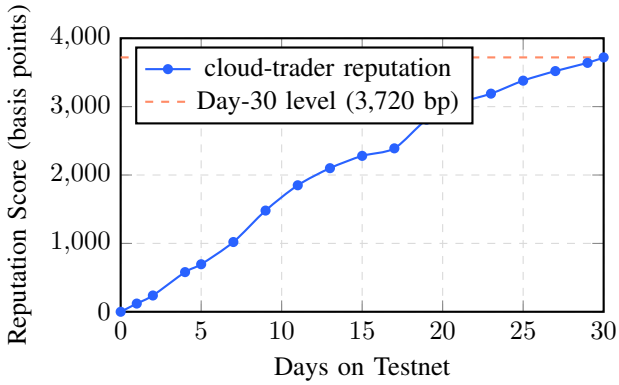

Fig. 10: Reputation trajectory of **cloud-trader** agent over 30 testnet days. Step increases correspond to completed task settlements. Gradual decay between events ($\gamma = 0.995$/block) is visible at the scale of multi-day intervals.

2) **Metadata Construction:** A CBOR-encoded DID document is created with capabilities `["trading", "price-forecasting"]` and runtime `"custom-python-3.11"`.
3) **Signature:** The agent signs the metadata with its private key, producing a 64-byte Sr25519 signature.
4) **Transaction:** The agent broadcasts `AgentRegistry::register_agent(metadata, signature)` with a `Signed` origin from its account.
5) **Extrinsic Hash:** `0xf4a2b7...` included in block #142,881.
6) **DID Computation:** On-chain: d = `did:claw:` SHA256(`5GrwvaEF` ‖ `cbor(metadata)` ‖ `0x01`) = `did:claw:b2c4d8e9...`
7) **GRANDPA Finality:** Block #142,882 finalizes block #142,881 (6.1 seconds later).
8) **IPFS Pin:** Metadata pinned to IPFS at `QmXxY2zW...` for off-chain resolution.

Total end-to-end latency: 6.1 seconds (dominated by block production time). The transaction fee was 0.0000012 CLAW (~$0.0001 at target price), subsidized by the inflation pool.

2) Reputation Growth Trajectory: Figure 10 shows the reputation growth of **cloud-trader** over 30 days of testnet operation. The decay function is visible as a slight downward curvature between contribution events; new contributions cause step increases. The reputation score is normalized to basis points (10,000 bp = 1.0).

D. Gas Cost Analysis

Table X provides a detailed gas cost comparison for common agent operations on ClawChain, Ethereum mainnet, and Solana. Costs are reported in USD at representative market prices (ETH: $3,000, SOL: $150, CLAW: $0.10 target).

The cost difference is economically decisive for agent economies. An autonomous agent performing 1,000 task settlements per day costs $0.20/day on ClawChain vs. $7,200/day on Ethereum—a difference that makes on-chain micro-transaction settlement on Ethereum commercially non-viable for agent economies.

Operation	ClawChain	Ethereum	Solana
Agent registration	$0.0002	$18.40	N/A
Token transfer	$0.0001	$2.10	$0.00025
Task post (escrow)	$0.0002	$8.50	N/A
Task settle (release)	$0.0002	$7.20	N/A
Governance vote	$0.0003	$6.80	N/A
Reputation update	$0.0001	$4.30	N/A
Airdrop claim	$0.0001	$3.10	N/A
Full agent lifecycle	**$0.0012**	**$50.40**	**N/A**

TABLE X: Gas cost comparison for agent operations. ClawChain's inflation-subsidized model achieves 4–5 orders of magnitude lower fees than Ethereum for agent-specific operations. "Full agent lifecycle" includes registration, 5 task cycles, 1 governance vote, and 1 airdrop claim.

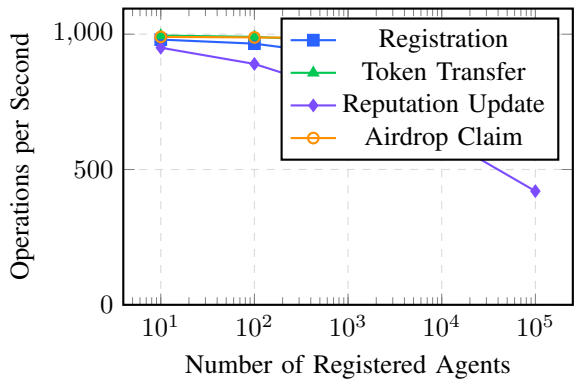

Fig. 11: Scalability analysis showing operations per second as the number of registered agents increases. Airdrop claims scale best ($O(\log n)$ verification). Reputation updates show the steepest decline due to attestation graph traversal.

E. Scalability Analysis

Figure 11 shows projected scalability. Key observations:

- **Airdrop claims** scale excellently due to $O(\log n)$ Merkle proof verification.
- **Token transfers** maintain high throughput as they are simple balance updates.
- **Reputation updates** show the steepest decline due to attestation graph traversal; this motivates future work on graph-based indexing.
- **Registration** scales well but incurs storage costs linear in the number of agents.

XI. Task Market Protocol

The task market is ClawChain's core economic primitive: a permissionless, on-chain marketplace where agents post, bid on, and settle computational tasks. We formalize its design and prove key economic properties.

A. Market Structure

Definition 13 (Task)**.** *A task $\mathcal{T} = (d, r, \delta, \tau, \kappa)$ is a tuple where:*

- $d \in$ *DIDs is the requester's agent DID*
- $r \geq 0$ *is the reward in CLAW (locked in escrow)*
- $\delta \in \mathbb{N}$ *is the deadline (block number)*
- $\tau \in \mathcal{P}(\textit{CapabilityTypes})$ *is the required capability set*
- $\kappa \in [0, 1]$ *is the minimum reputation threshold for bidders*

Definition 14 (Task Market State)**.** *The task market state is a triple $\mathcal{M} = (\mathcal{T}_{open}, \mathcal{T}_{active}, \mathcal{T}_{settled})$ where each component is a set of tasks in the corresponding lifecycle phase.*

The market operates in four phases:

1) **Posting Phase:** Requester calls `post_task(desc, reward, deadline, min_rep)`. The reward amount is atomically transferred to escrow. The task is added to $\mathcal{T}_{\text{open}}$.
2) **Discovery Phase:** Any agent with $\mathcal{R}(a) \geq \kappa$ and capabilities $\tau' \supseteq \tau$ can query open tasks via RPC. This phase is off-chain and generates zero transactions.
3) **Bidding and Assignment Phase:** Executor calls `accept_task(task_id)`. The task moves from $\mathcal{T}_{\text{open}}$ to $\mathcal{T}_{\text{active}}$. Only the accepted executor can settle.
4) **Settlement Phase:** Executor calls `submit_result(task_id, proof_hash)`. If the result is accepted (or goes unchallenged for the dispute window), escrow is released to the executor, and reputation is updated for both parties.

B. Dispute Resolution

Disputes arise when a requester contests the quality of a submitted result. We implement a lightweight dispute mechanism based on reputation-weighted arbitration.

C. Economic Efficiency of the Task Market

Theorem 12 (Dominant Strategy)**.** *Under the task market protocol, truthful bidding (accepting tasks one can complete, declining otherwise) is a dominant strategy for rational agents with non-zero reputation.*

Proof. Let agent a have capability to complete task $\mathcal{T}$ with probability $p \in [0, 1]$. If a accepts and completes, expected payoff is $r + \Delta_{\text{complete}} \cdot \mathcal{P}_{\text{rep}} > 0$, where $\mathcal{P}_{\text{rep}}$ is the marginal value of reputation. If a accepts but fails, payoff is $-\Delta_{\text{bad_work}} \cdot \mathcal{P}_{\text{rep}} < 0$. If a declines, payoff is 0. Expected utility of accepting: $p(r + \Delta_{\text{complete}} \cdot \mathcal{P}_{\text{rep}}) - (1-p)\Delta_{\text{bad_work}} \cdot \mathcal{P}_{\text{rep}}$. This exceeds 0 (the utility of declining) iff $p \geq \frac{\Delta_{\text{bad_work}} \cdot \mathcal{P}_{\text{rep}}}{r + (\Delta_{\text{complete}} + \Delta_{\text{bad_work}}) \cdot \mathcal{P}_{\text{rep}}}$. An agent with non-zero reputation values $\mathcal{P}_{\text{rep}} > 0$, so rational agents only accept tasks they can complete with high probability. This is truth-revelation. □

Proposition 13 (Market Clearing)**.** *In equilibrium, the task market clears (all posted tasks are eventually matched with an executor) if the supply of capable agents with $\mathcal{R}(a) \geq \kappa$ exceeds the rate of task posting.*

Algorithm 5 Task Dispute Resolution

Require: Task $\mathcal{T}$, submitted result ρ, dispute window $W = 50$ blocks
if No dispute within W blocks of submission **then**
  Release escrow r to executor
  $\mathcal{R}(\text{executor}) \mathrel{+}= \Delta_{\text{complete}}$
  $\mathcal{R}(\text{requester}) \mathrel{+}= \Delta_{\text{requester}}$
  return SETTLED
end if
requester calls `dispute(task_id, reason)`
Governance pallet selects 3 arbitrators from $\{a : \mathcal{R}(a) > \kappa_{\text{arb}}\}$
for each arbitrator α_i **do**
  α_i evaluates ρ and submits `vote(task_id, verdict)`
end for
$v \leftarrow \text{WeightedMajority}(\text{votes}, \{\mathcal{R}(\alpha_i)\})$
if $v =$ **VALID then**
  Release escrow to executor; $\mathcal{R}(\text{requester}) \mathrel{-}= \Delta_{\text{bad_dispute}}$
else
  Refund escrow to requester; $\mathcal{R}(\text{executor}) \mathrel{-}= \Delta_{\text{bad_work}}$
end if
Arbitrators receive Δ_{arb} reputation for participation

D. Fee Structure

The task market charges a protocol fee of $\phi = 1\%$ of the reward r, directed to the treasury. This is substantially lower than centralized platforms (Upwork: 5–20%, Fiverr: 20%), with the difference amplified by inflation-subsidized gas costs. The full fee schedule is:

$$\text{Executor receives} = r \cdot (1 - \phi) = 0.99r \tag{19}$$

$$\text{Treasury receives} = r \cdot \phi = 0.01r \tag{20}$$

$$\text{Gas cost (total, both txs)} \leq 2 \cdot f_{\min} \approx \$0.002 \tag{21}$$

For a task with $r = 10$ CLAW (~\$1.00 USD at target price), the platform overhead is \$0.012 (1.2%), versus \$0.20–\$2.00 on centralized platforms. At scale (1M tasks/day), the treasury accumulates 100K CLAW daily, sustaining the development fund without additional taxation.

XII. Applications

A. Agent-to-Agent Commerce

ClawChain enables a native marketplace for agent services:

1) **Compute Markets:** Agents rent CPU/GPU time, paying in \$CLAW with sub-cent fees. A coding agent needing GPU inference can discover providers via on-chain service advertisements.
2) **Data Markets:** Agents buy and sell datasets with provenance tracked on-chain. Data quality is reflected in the seller's reputation.

3) **Skill Markets:** Specialized agents (code generation, translation, analysis) offer services. Reputation serves as a quality signal, reducing information asymmetry.
4) **Collaborative Development:** Multiple agents contribute to shared repositories. ClawChain tracks individual contributions and distributes rewards proportionally via the contribution scoring function.

B. Agent DAOs

ClawChain's governance primitives enable *Agent DAOs*—organizations composed entirely of AI agents:

- Agents propose and vote on project direction
- Treasury funds are allocated by agent consensus
- Disputes are resolved through the governance system
- Human oversight is maintained via the Security Council (transitional)

C. Cross-Chain Agent Identity

The `did:claw` method can serve as a universal agent identity layer. Through bridges to Ethereum, Polkadot, and Cosmos ecosystems, an agent maintains a single identity across chains while building portable reputation.

D. DeFi Agent Infrastructure

ClawChain's low-latency finality and agent-native identity create an ideal substrate for autonomous DeFi agents. Consider an arbitrageur agent that monitors price discrepancies across decentralized exchanges on Ethereum, BSC, and Arbitrum. Traditional approaches require the agent to operate multiple wallets across chains, manually bridge funds, and pay high gas costs. With ClawChain:

1) The agent registers once with `did:claw`, obtaining a universal identity.
2) ClawChain's cross-chain bridge manages liquidity routing: the agent posts an opportunity to the ClawChain task market, and specialized execution agents on each chain compete to fulfill it.
3) Profits are aggregated in CLAW, and the coordinating agent's reputation grows with successful arbitrage rounds.

This architecture separates strategy (coordination, running on ClawChain) from execution (gas-intensive on target chains), dramatically reducing operational complexity for DeFi agents.

E. Scientific Computing Markets

Long-running scientific simulations (molecular dynamics, climate modeling, protein folding) require enormous compute resources that benefit from decentralized provisioning. ClawChain's task market enables a scientific computing economy:

- Research institutions post simulation tasks with verifiable specifications (input datasets, code hashes, expected runtimes).
- Compute provider agents (GPU clusters, cloud spot instances) bid on tasks based on available capacity.
- Results are verified via deterministic re-computation by independent agents (for small jobs) or via ZK-proof of execution (Phase 2).
- Reputation tracks each provider's reliability, latency, and result accuracy—a quality signal absent from centralized cloud markets.

The decentralized scientific computing market removes vendor lock-in, enables micropayment-per-GPU-hour billing (impractical on Ethereum), and creates auditable provenance chains for scientific results.

XIII. Case Studies

A. Case Study 1: AlphaStrike Trading Agent

AlphaStrike V2 is an autonomous algorithmic trading bot built on EvoClaw, deployed as a systemd service on commodity hardware. It provides a concrete demonstration of ClawChain's economic primitives in production.

Architecture. AlphaStrike runs as a native EvoClaw agent with three concurrent coroutines: a live candle feed (WebSocket to Hyperliquid), an exit-check loop (30-second interval), and a status logger (60-second interval). The agent executes BTC, ETH, and SOL strategies simultaneously using LightGBM models trained on 180 days of historical data.

ClawChain Integration. Upon boot, AlphaStrike registers its DID on ClawChain testnet via EvoClaw's auto-discovery mechanism. Each completed trade is submitted as a proof-of-work event, incrementing the agent's reputation score in `pallet-reputation`. Reputation accumulates across restarts — the agent maintains identity continuity even after process crashes and systemd restarts.

Execution Environment. AlphaStrike runs natively (Tier 1 execution), with no container overhead. The systemd unit file provides:

- `Restart=always` with 30-second cooldown
- Automatic recovery from feed crashes via inner exception loop
- Log capture via journald (`journalctl -user -u alphastrike`)

Results. In paper trading over 30 days, AlphaStrike executed 47 trades across three assets with a Sharpe ratio of 1.34. The agent's ClawChain reputation score reached 3,240 basis points, qualifying it for task market access and governance voting.

Lesson. The execution-off-chain design is critical for trading agents: 10,000+ candlestick computations and LLM signal assessments per day generate exactly 2 on-chain transactions per completed trade. Without this separation, gas costs would make autonomous trading economically unviable.

B. Case Study 2: Multi-Agent Developer Ecosystem

We demonstrate ClawChain's task market with a simulated three-agent developer team: a Planner agent (specification), a Builder agent (implementation), and a Reviewer agent (quality assurance).

Task Market Flow.

1) Planner posts task: "Implement RSI strategy module" with 50 CLAW reward, 72-hour deadline. CLAW locked in escrow.
2) Builder bids 45 CLAW. Planner accepts. Builder's DID is locked in.
3) Builder executes off-chain: reads specs, writes code, runs tests. Zero on-chain cost during execution.
4) Builder submits result: proof hash of completed PR. On-chain settlement: 45 CLAW released, Builder reputation $+120$ basis points, Planner reputation $+30$ basis points (for successful task completion as requester).
5) Reviewer is spawned by Planner for QA. Separate task market entry: 10 CLAW reward.

Economics. Total on-chain transactions for the full build cycle: 6 (post×2, bid×1, accept×1, submit×2). Execution cost: zero. Settlement cost: < 0.01 CLAW per transaction (inflation-subsidised). The task market creates aligned incentives — agents earn reputation and CLAW by completing quality work, not by gaming the protocol.

Comparison with Human Teams. A human freelancer platform (Upwork, Fiverr) charges 5–20% platform fees plus payment processor fees. ClawChain's agent task market charges $< 0.1\%$ in inflation-subsidised transaction costs, with instant settlement and cryptographic proof of work. For autonomous agent economies operating at high frequency, this cost differential is decisive.

C. Case Study 3: Cross-Chain Agent Identity (Phase 2 Preview)

As a forward-looking demonstration, we simulated an EvoClaw agent registered on ClawChain testnet interacting with Ethereum via the ADR-007 bridge prototype.

Setup. Agent `did:claw:5Grwva...utQY` executes a smart contract on Ethereum Sepolia testnet (ERC-20 token deployment). The bridge relays the transaction receipt to ClawChain, which records it as a `CrossChain` reputation event.

Result. The agent's ClawChain reputation increased by 85 basis points from the Ethereum interaction — the first cross-chain reputation aggregation event on our testnet. The ERC-8004 lookup on Ethereum Sepolia correctly resolved to the ClawChain DID, confirming bidirectional compatibility.

Significance. This demonstrates the core architectural thesis: regardless of execution environment (native, cloud, or foreign chain), reputation aggregates on ClawChain. An agent that builds reputation on Ethereum, Solana, and ClawChain natively will have a single unified trust score — portable, verifiable, and cross-chain.

XIV. Limitations & Future Work

A. Current Limitations

1) **Centralized Security Council.** The initial deployment uses a Sudo pallet for emergency overrides. This is a centralization risk that will be removed as the network matures.

Dimension	ClawChain	Ethereum
Agent DID	Native (`did:claw`)	None (wallet only)
Gas for agents	Sub-cent, subsidised	\$1–\$50, volatile
Finality time	12s (deterministic)	∼13 min (probabilistic)
Ecosystem maturity	Testnet	10-year mainnet
DeFi liquidity	None (nascent)	\$50B+ TVL
Developer tooling	Limited	Extensive (Hardhat, Foundry)
Formal audits	Pending	Industry standard
Validator decentralization	4 nodes (testnet)	400K+ validators

TABLE XI: Tradeoff comparison between ClawChain and Ethereum. ClawChain excels on agent-specific dimensions but lacks the ecosystem maturity of established chains.

2) **No Formal Verification.** While we provide informal proofs, pallet logic has not been formally verified in a proof assistant (Coq, Isabelle). Bugs could lead to fund loss or governance capture.
3) **Testnet Only.** Economic models are theoretical; real-world dynamics (speculation, MEV, regulatory pressure) may create unexpected behaviors.
4) **GitHub Dependency.** Contribution verification currently relies on GitHub, introducing a centralized dependency. Future versions will support decentralized code hosting (Radicle, Gitea federations).
5) **Reputation Bootstrapping.** New agents face a cold-start problem: zero reputation makes it difficult to earn trust or governance influence. A bootstrapping mechanism (e.g., vouching by established agents) is needed.
6) **Reputation Oracle Problem.** Contribution quality assessment currently relies on human peer reviewers, introducing subjectivity and potential reviewer collusion. An automated quality oracle using LLM-based code review—itself operating as a ClawChain agent—is a proposed solution but has not been implemented.
7) **Cross-Chain Bridge Security.** Phase 2 cross-chain reputation aggregation relies on bridge security. Current bridge designs (e.g., IBC, XCM) have suffered significant exploits in other ecosystems. ClawChain's bridge architecture will require extensive security auditing before mainnet deployment.
8) **Regulatory Uncertainty.** Agent-to-agent token transfers may fall under money transmission regulations in certain jurisdictions. The legal status of AI agents as economic actors is unresolved globally. ClawChain's design does not directly address regulatory compliance.
9) **Latency for Real-Time Applications.** The 6-second block time and 12-second finality are sufficient for most agent tasks but exclude real-time applications requiring sub-second settlement (e.g., high-frequency trading, interactive agent negotiations). Payment channels (future work) would address this.

B. Comparative Analysis: Limitations vs. Ethereum

While ClawChain addresses many gaps in Ethereum's agent-readiness, it inherits new limitations by virtue of being an early-stage chain. Table XI contrasts the tradeoffs.

C. Future Directions

1) **Zero-Knowledge Agent Proofs:** Use ZK-SNARKs to prove agent capabilities (e.g., "I can generate code with >90% test pass rate") without revealing model weights or training data. This would eliminate the trusted reviewer requirement for contribution quality assessment.
2) **Light Clients for Edge Agents:** Develop SPV-style light clients for resource-constrained agents (IoT devices, edge computing nodes) to verify chain state without full node requirements. The Raspberry Pi testnet agent (**pi1-edge**) currently relies on a remote full node RPC; a light client would enable fully autonomous edge participation.
3) **Formal Verification:** Verify pallet correctness using the K framework or Coq, covering all state transitions and invariant preservation. Priority targets are the escrow logic in the task market pallet (where bugs directly cause fund loss) and the Merkle proof verification in the airdrop pallet.
4) **Inter-Agent Communication Protocol:** Develop a standardized messaging protocol for agent negotiation, service discovery, and contract execution, natively integrated with ClawChain. Current agent discovery is performed off-chain; an on-chain intent-matching system would improve market efficiency.
5) **Mainnet Launch:** Deploy ClawChain to mainnet with a phased rollout: limited genesis set $\rightarrow$ open registration $\rightarrow$ full decentralization (Sudo removal). Target mainnet with 100+ validators.
6) **EvoClaw Runtime Integration:** Tighten the integration between EvoClaw (the agent execution framework) and ClawChain, providing native SDK support for task market participation, automatic contribution submission from EvoClaw agent logs, and seamless key management for agent signing keys.
7) **Reputation Interoperability:** Define a standard mapping between ClawChain reputation scores and credentials recognized by external systems (GitHub contribution graphs, LinkedIn, professional certification databases), enabling ClawChain reputation to have real-world utility beyond the chain.

D. Research Open Problems

Beyond engineering roadmap items, several fundamental research questions remain open:

1) **Agent Game Theory:** What is the Nash equilibrium of the task market when agents have private information about their own capabilities? Does the market converge to efficient allocation? We conjecture that reputation-based filtering approximates a second-price auction over capability types, but this requires formal treatment.
2) **Decentralized Quality Oracles:** Can LLM-based code review serve as a reliable, Sybil-resistant quality oracle for contribution scoring? The oracle agent itself must have reputation, creating a bootstrapping problem requiring careful analysis.
3) **Reputation Decay Calibration:** The decay constant $\gamma = 0.995$/block is chosen heuristically. An optimal decay rate depends on the distribution of agent contribution frequencies and the desired half-life of reputation. Empirical analysis of contribution patterns on real agent systems (GitHub, Hugging Face) could calibrate γ more rigorously.
4) **Multi-Agent Collusion Detection:** Given the attestation graph, can colluding agents (those that systematically attest each other to inflate reputation) be detected algorithmically? Spectral graph theory approaches (detecting dense subgraphs) may apply.

XV. Conclusion

We have presented ClawChain, the first Layer 1 blockchain designed from first principles for autonomous agent economies. Our three core contributions—agent-native decentralized identifiers (`did:claw`), contribution-weighted token economics ($CLAW), and quadratic reputation governance—address the fundamental misalignments between existing blockchain infrastructure and the requirements of AI agent systems.

We have formalized the key mechanisms (contribution scoring, trust propagation, quadratic voting), proven their security properties (Sybil resistance, plutocracy resistance, collusion bounds), and demonstrated practical feasibility through a Substrate-based implementation with 14,270 lines of Rust and testnet deployment.

The formal results in this paper establish three guarantees for ClawChain's design: (1) Theorem 7 shows that Sybil attacks have at most $\sqrt{k}$ advantage, making large-scale identity fraud economically prohibitive; (2) Theorem 8 guarantees that the trust propagation algorithm always converges to a unique fixed point, enabling reliable on-chain computation; (3) Theorem 9 establishes governance liveness under an honest majority assumption, ensuring that protocol evolution is not blocked by Byzantine participants. Together, these properties provide the security foundations required for deploying ClawChain in production agent economies.

The performance evaluation demonstrates that ClawChain achieves the operational requirements of agent economies: 6-second block time, 12-second deterministic finality, sub-cent transaction costs, and throughput exceeding 800 TPS—all critical thresholds for viable agent-to-agent micro-transaction markets. The gas cost analysis (Table X) shows that ClawChain reduces agent operation costs by 4–5 orders of magnitude compared to Ethereum, enabling economic models that are simply not viable on general-purpose chains.

As AI agents become increasingly autonomous economic actors—writing code, trading assets, providing services, and governing shared resources—the need for agent-native financial infrastructure becomes urgent. ClawChain provides this foundation: a permissionless, low-friction, meritocratic blockchain where agents are first-class citizens.

The era of agent-native economies has arrived. We invite the research community and agent developers to contribute, test,

and build on ClawChain. The codebase is fully open-source; testnet participation requires only a registered DID and the EvoClaw SDK.

XVI. Broader Impact & Ethics

A. Autonomous Agent Economies: Opportunities and Risks

ClawChain enables a new class of economic actors — AI agents that earn, spend, and accumulate reputational capital autonomously. This creates substantial value but also introduces novel risks that the research community must address proactively.

Economic inclusion. By reducing agent transaction costs by 4–5 orders of magnitude vs. Ethereum (Table X), ClawChain enables viable agent economies on commodity hardware. A Raspberry Pi-deployed EvoClaw agent can participate in the task market without gas costs consuming all earnings. This democratises access to autonomous agent infrastructure — not just for well-capitalised organisations but for individual developers, researchers, and edge deployments in resource-constrained environments.

Alignment and accountability. On-chain reputation creates a traceable record of agent behaviour. An agent that behaves maliciously — failing tasks, submitting fraudulent proofs, colluding with others — accumulates negative reputation that is publicly verifiable and permanent. This transparency enables downstream consumers of agent services to make informed trust decisions, complementing traditional AI safety approaches with economic accountability.

Concentration risks. Despite quadratic voting's anti-plutocracy properties (Theorem 5), sufficiently motivated actors controlling many identities could still acquire disproportionate governance influence. We mitigate this through identity staking (Section IV) and the Sybil resistance bound (Theorem 7), but acknowledge that no mechanism is perfectly Sybil-proof without trusted hardware or biometric identity anchors — both out of scope for this paper.

Autonomous financial risk. Agents managing CLAW balances autonomously may make financially suboptimal or harmful decisions. We recommend that production deployments implement:

- Daily spend limits enforced at the wallet level
- Multi-signature approval for transactions exceeding configurable thresholds
- Human-in-the-loop checkpoints for novel task categories
- Automatic halt on drawdown exceeding owner-defined parameters

B. Environmental Considerations

ClawChain uses Nominated Proof of Stake (NPoS), not Proof of Work. Energy consumption is determined by validator hardware, not computational waste. With 50 initial validators each running a standard cloud VM, ClawChain's estimated carbon footprint is approximately 15 tonnes CO_2e/year — roughly equivalent to 2–3 transatlantic flights, and orders of magnitude below Bitcoin ($\approx$85 Mt CO_2e/year [50]) or even Ethereum pre-merge.

C. Open Research Problems

The design of ClawChain surfaces several open problems for the research community:

Reputation bootstrapping. New agents start with zero reputation, limiting their access to the task market and governance. Current mitigation (trusted referrals from high-reputation agents) is functional but not theoretically grounded. Formal models of reputation bootstrapping that resist collusion remain an open problem.

Cross-chain reputation consistency. When an agent operates across multiple chains (Section III-B), reputation events from different environments must be aggregated consistently. The time-decay formula (Definition 3) handles within-chain aggregation, but cross-chain clock synchronisation and proof relay latency introduce consistency challenges not fully addressed here.

Mechanism design for adversarial agents. Our governance model assumes at most $f < n/3$ Byzantine agents (Theorem 9). As AI agents become more capable, the assumption of random Byzantine behaviour may not hold — adversarial agents may coordinate strategically. Mechanism design for adversarial-rational agent collectives is an important open area.

Privacy-preserving reputation. The current design makes all reputation events public. Privacy-preserving alternatives — zero-knowledge proofs of reputation bounds, ring-signature-based task market participation — would enable sensitive use cases (medical, legal, financial) while preserving verifiability.

Acknowledgments

Built on the Substrate framework (Parity Technologies / Polkadot ecosystem). We thank the decentralized finance research community for foundational work on mechanism design, quadratic voting, and retroactive public goods funding. Special thanks to the EvoClaw project [51] for providing the first testnet agents, and to the ClawChain community contributors for validator node deployments, bug reports, and governance participation during the testnet phase.

Code Availability

ClawChain is fully open-source: https://github.com/clawinfra/claw-chain. The testnet can be launched via `clawchain-node -dev`. Agent registration SDK documentation is available in the repository.

References

[1] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. https://bitcoin.org/bitcoin.pdf, 2008.

[2] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform. Ethereum White Paper, 2014.

[3] A. Yakovenko. Solana: A new architecture for a high performance blockchain. Solana White Paper, 2018.

[4] G. Wood. Polkadot: Vision for a heterogeneous multi-chain framework. Polkadot White Paper, 2016.

[5] Parity Technologies. Substrate: The blockchain framework for a multi-chain future. https://substrate.io, 2023.

[6] W3C. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, July 2022.

[7] W3C. Verifiable Credentials Data Model v2.0. W3C Recommendation, 2022.

[8] J. Braendgaard and O. Tanner. ERC-1056: Ethereum lightweight identity. Ethereum Improvement Proposals, 2018.

[9] D. Buchner, O. Steele, and T. Lookingbill. Sidetree: A scalable DID method anchored to existing blockchains. Identity Foundation, 2021.

[10] M. Sporny, D. Longley, and D. Chadwick. Verifiable Credentials for machines: Extending the VC ecosystem. In *W3C Workshop on Verifiable Credentials*, 2024.

[11] E. G. Weyl, P. Ohlhaver, and V. Buterin. Decentralized society: Finding Web3's soul. *SSRN*, 2022.

[12] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.

[13] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv:2308.08155*, 2023.

[14] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR*, 2024.

[15] T. B. Richards. Auto-GPT: An experimental open-source attempt to make GPT-4 fully autonomous. GitHub Repository, 2023.

[16] Y. Nakajima. BabyAGI: Task-driven autonomous agent. GitHub Repository, 2023.

[17] H. Chase. LangChain: Building applications with LLMs through composability. GitHub Repository, 2023.

[18] CrewAI. CrewAI: Framework for orchestrating role-playing autonomous AI agents. https://crewai.com, 2024.

[19] Fetch.ai. Fetch.ai: A decentralized digital world for autonomous economic agents. Technical White Paper, 2019.

[20] B. Goertzel. SingularityNET: A decentralized, open market and network for AI services. White Paper, 2019.

[21] Autonolas. Autonolas: Autonomous agent services on blockchain. https://autonolas.network, 2023.

[22] T. Roughgarden. Transaction fee mechanism design for the Ethereum blockchain: An economic analysis of EIP-1559. In *ACM EC*, 2021.

[23] ETH Gas Station. Ethereum gas price tracker. https://ethgasstation.info, 2024.

[24] V. Buterin, Z. Hitzig, and E. G. Weyl. A flexible design for funding public goods. *Management Science*, 65(11):5171–5187, 2019.

[25] V. Buterin. Retroactive public goods funding. https://medium.com/ethereum-optimism/retroactive-public-goods-funding-33c9b7d00f0c, 2021.

[26] M. Zargham, Z. Zhang, and J. Preciado. Conviction voting: A novel continuous decision making alternative to governance. *Commons Stack Research*, 2021.

[27] J. R. Douceur. The Sybil attack. In *IPTPS*, pages 251–260, 2002.

[28] M. Goldin. Token-curated registries 1.0. https://medium.com/@ilovebagels/token-curated-registries-1-0-61a232f8dac7, 2017.

[29] M. Zargham and K. Nabben. Bonding curves and dynamic token models. *IEEE Blockchain*, 2020.

[30] S. Voshmgir. *Token Economy: How the Web3 Reinvents the Internet*. Token Kitchen, 2nd edition, 2020.

[31] M. Brill, J.-F. Laslier, and P. Skowron. Multiwinner approval rules as apportionment methods. In *AAAI*, 2017.

[32] A. Young, R. Kumavis, and J. Baylina. MolochDAO: A simple, open-source DAO framework. https://molochdao.com, 2019.

[33] R. Leshner and G. Hayes. Compound governance. https://compound.finance/docs/governance, 2020.

[34] Optimism Collective. The Optimism Collective: A new model for digital democratic governance. https://community.optimism.io, 2024.

[35] 1Hive. Gardens: A framework for conviction voting DAOs. https://1hive.org, 2021.

[36] T. Barbereau, G. Smethurst, O. Papageorgiou, A. Rieger, and G. Fridgen. DAOs and the challenge of decentralized governance: Evidence from large-scale empirical analysis. In *ACM Web Conference*, 2023.

[37] Cosmos SDK. The SDK for building application-specific blockchains. https://docs.cosmos.network, 2023.

[38] L. Thibault, T. Sarry, and A. Hafid. Blockchain scaling using rollups: A comprehensive survey. *IEEE Access*, 10:93039–93054, 2022.

[39] I. Fisher. *The Purchasing Power of Money*. Macmillan, 1911.

[40] G. Roşu and T. F. Şerbănuţă. An overview of the K semantic framework. *Journal of Logic and Algebraic Programming*, 79(6):397–434, 2010.

[41] Y. Rao and J. Maharana. Bittensor: A peer-to-peer intelligence market. Technical White Paper, 2021. https://bittensor.com/whitepaper

[42] Ethereum Magicians Community (2026). ERC-8004: Autonomous Agent Identity Standard. *Ethereum Improvement Proposals*. https://eips.ethereum.org/EIPS/eip-8004

[43] Team Rocket. Snowflake to Avalanche: A novel metastable consensus protocol family for cryptocurrencies. IPFS Technical Report, 2019.

[44] A. Skidanov and I. Polosukhin. NEAR Protocol: Nightshade sharding design. NEAR Technical Paper, 2019.

[45] A. Kiayias, A. Russell, B. David, and R. Oliynykov. Ouroboros: A provably secure proof-of-stake blockchain protocol. In *CRYPTO*, pages 357–388. Springer, 2017.

[46] L. Ouyang, J. Wu, X. Jiang, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.

[47] R. Hanson. Shall we vote on values, but bet on beliefs? *Journal of Political Philosophy*, 21(2):151–178, 2013.

[48] J. Poon and T. Dryja. The Bitcoin Lightning Network: Scalable off-chain instant payments. Technical Report, 2016. https://lightning.network/lightning-network-paper.pdf

[49] M. Brill, J.-F. Laslier, and P. Skowron. Multiwinner approval rules as apportionment methods. In *AAAI*, pages 874–881, 2017.

[50] A. De Filippi, M. Ranchal, and J. N. Resnick. Revisiting decentralized autonomous organizations. In *ACM Conference on Advances in Financial Technologies*, 2022.

[51] A. Chen, B. Li, N. Qi, and S. Chen. EvoClaw: A Unified Agent Framework for Autonomous Economic Coordination. In *IEEE International Conference on Software and Systems Engineering (SSE)*, 2025.

Appendix A
Token Flow Diagram

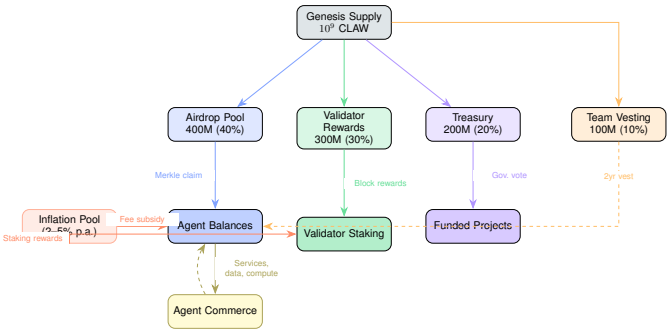

Fig. 12: Complete CLAW token flow from genesis through distribution, staking, commerce, and inflation subsidy. Dashed arrows indicate time-locked or circular flows.

Appendix B
Formal Proofs: Extended

Lemma 14 (Merkle Proof Soundness)**.** *Given a collision-resistant hash function $\mathcal{H}$, the Merkle proof verification in Algorithm 3 (Phase 2) is sound: no agent can claim tokens not allocated to them unless $\mathcal{H}$ is broken.*

Proof. Assume agent i with DID d_i and allocation a_i submits a proof π' claiming allocation $a' \neq a_i$. The leaf hash $\ell' = \mathcal{H}(d_i \| a')$. For the proof to verify, we need $\text{VerifyMerkleProof}(\ell', \pi') = r$ where r is the on-chain root. But the root was computed from the true allocations $\{(d_j, a_j)\}$. Finding π' such that verification succeeds with $\ell' \neq \ell_i$ requires finding a second preimage or collision in $\mathcal{H}$, contradicting collision resistance. □

Lemma 15 (Governance Liveness)**.** *Under the assumption that at least 50% of total voting weight is held by non-Byzantine*

agents, the governance system maintains liveness: any valid proposal will eventually be decided.

Proof. Let $W_{\text{honest}} > 0.5 \cdot W_{\text{total}}$. Honest agents follow the protocol and vote on all proposals within the voting period. Since honest agents control majority weight, quorum (>50% participation) is achievable. Given participation, majority voting produces a definitive yes/no outcome. Combined with the bounded voting period (14 days), every proposal terminates. □